The Frohmanian Tether on SDiff($\mathbb{T}^3$): A Metriplectic Correction, Its Canonicity, and an Obstruction to Damping

Conditional statements verified in Lean 4

Benjamin Stanley Frohman

Independent Researcher, Texas, United States

frohmanbenjamin@gmail.com ORCID 0009-0006-7068-3718

Companion repository: https://github.com/BenFrohman/frohmanian-tether-sdiff

Version 0.1.4; this version's DOI: (to be assigned on publication)

Preprint. Work in progress. All main results are conditional; see Section 1.1.

10 November 2026

Abstract

We describe a Lean 4/Mathlib formalization of the *Frohmanian tether*, a symmetric quadratic correction $T(F,G) = -\kappa \|\omega\|_{L^2}^2 \int \langle \Pi_u \, \delta F/\delta\omega, \Pi_u \, \delta G/\delta\omega \rangle$ added to the Arnold Lie–Poisson bracket of ideal fluids, where Π_u is the L^2-orthogonal projection away from the velocity u and $\kappa = 3/2$. The library proves: the negative-feedback condition (C3) unconditionally; a uniqueness ("canonicity") statement inside a restricted kernel class; and an *obstruction to damping*: every symmetric kernel B satisfying the kinetic-energy degeneracy (C2) has $B(H,M) = B(M,H) = 0$ for every functional M, so such a kernel contributes no damping to any functional, in particular not to $\|\omega\|_{L^\infty}$. (C2) is not proved for T. (C1) with the functionals held fixed is refuted for T under a concrete group of measure-preserving maps, using a translation; the *transported* (equivariant) form of (C1), in which the functionals move with the vorticity, $T(g\cdot\omega; F\circ g^{-1}, G\circ g^{-1}) = T(\omega; F, G)$, is formalized and *proved* for T under all translations (via translation equivariance of the functional gradient and of Biot–Savart); for rotations and general volume-preserving diffeomorphisms it is open. Using a new family of smooth, compactly supported, divergence-free test fields $\nabla \times (\chi e_2)$ concentrated at any point with any radius, the library proves that T satisfies the corrected weight condition of its canonicity theorem (every locally integrable density of T equals $-\kappa\|\omega\|^2$ almost everywhere). Version 0.1.2 removes every hypothesis of earlier versions that turned out to be false or vacuous, with Lean refutations where possible: $\delta H/\delta\omega = u$, transitivity of the action on Casimir levels, the pointwise strain bound $\|\nabla u(x)\| \le C\|\omega\|_{L^\infty}$, the invariance hypothesis `TetherIntegralPreserved`, a circular Jacobi lemma, an antisymmetric uniqueness clause that can hold only for $T \equiv 0$, and a uniform lower bound on local existence times; it also repairs the definition of a physical solution, which in earlier versions admitted only solutions constant in time. A further pass rebuilds the functional derivative as the unique divergence-free L^2 gradient along smooth compactly supported test fields (so it is a genuine, choice-free derivative), proves that T is nondegenerate, refutes the unrestricted polarization identity for T and the pointwise weight hypothesis of the canonicity theorem, refutes for T the almost-everywhere weight hypothesis over all (possibly non-measurable) densities and restricts it to locally integrable densities, and restricts the Kato local-existence witness to its own time interval, which makes the continuation from local to global solutions an explicit hypothesis. On the analytic side the library proves a scalar comparison lemma for $y' \le Cy^2 - \kappa' y^3$ and several conditional implications of the form "(cubic rate or uniform vorticity bound on local solutions) + (Kato local existence) + (Kato–Beale–Kato–Majda continuation) + (uniqueness) ⇒ global smooth solution", in which every analytic input is an explicit hypothesis. The rate and closing-bound hypotheses are open, are not supplied by the

tether (its damping coefficient is 0), and are of the same strength as the global regularity problem. Version 0.1.4 adds two explicit moving exact solutions, the decaying shear flow $e^{-\nu k^2 t}(a \sin kx_2, 0, 0)$ and the decaying ABC (Beltrami) flow with pressure $-|u|^2/2$, proved nonzero and to satisfy the library's Navier–Stokes equations for all $t \geq 0$, and, for $k = 2\pi m$, to be global smooth periodic solutions in the class of the periodic Clay statement for these particular data; it also proves that no continuous nonzero $\mathbb{Z}^3$-periodic field has finite energy on $\mathbb{R}^3$, so these flows are *not* Kato witnesses (`KatoLocalWitness` is empty for all nonzero periodic data) and the $\mathbb{R}^3$-class hypotheses hold only vacuously there. **Every main result is conditional.** The open hypotheses, by Lean name, are: `KatoLocalWitness` (hKato), `BKMGlobalContinuation`, `SmoothPhysicalUniqueness`, `PeriodicBKMContinuation`, `KatoUniquenessOnExistenceTimes`, `IsCoadjointMotion` (classical results not in Mathlib); `LocalTetherRate`, `LocalTetherBound`, `LocalTetherClosingBound`, `PeriodicTetherClosingBound`, `PeriodicDirectionDepletionBound` and the global-path forms `FrohmanianTetherRate`, `FrohmanianTetherBound`, `FrohmanianTetherClosingBound`, `RiccatiMajorantExists` (as strong as the open problem); the kernel-class conditions on a competitor B in the canonicity theorems; and, for T itself, `DegenerateWRTKineticEnergy` (C2) at regular vorticities. **This work does not prove global regularity of the three-dimensional Navier–Stokes equations and makes no claim toward the Clay Millennium Problem.** The build (Lean `v4.30.0-rc1`, Mathlib `v4.30.0-rc1`, 2795 jobs) contains 760 theorem constants (670 stated in the sources and 90 auto-generated equation, congruence and injectivity lemmas) and 1074 declarations in 33 modules, with no `sorry`, no `native_decide`, no axiom declarations, and every declaration depending only on `propext`, `Classical.choice` and `Quot.sound`; this is enforced in continuous integration and reproduced from a fresh clone by the single command `bash scripts/verify.sh` (or `make verify`).

Contents

1 Introduction and scope **3**
1.1 The results are conditional 4

2 The formal setting **5**

3 The tether kernel and conditions (C1)–(C3) **6**
3.1 Transported (equivariant) (C1) 7
3.2 The identification $\delta H/\delta\omega = u$ is false (removed) 7
3.3 Other removed hypotheses (version 0.1.2) 8

4 The obstruction to damping **10**

5 Canonicity **10**

6 Concentrated divergence-free test fields and the weight condition **12**

7 Bracket structure and Casimirs **12**

8 The analytic layer **13**

9 The periodic Clay statement and a conditional reduction **14**

10 Explicit exact solutions: decaying shear and ABC flows **15**

11 What the Lean development does and does not prove **16**
11.1 Remaining assumptions, by category 17
11.2 Mathlib declarations used; project-local definitions 18

A Exact Lean statements **19**
A.1 Setting . . . 19
A.2 Tether, (C1)–(C3), obstruction, canonicity . . . 21
A.3 Concentrated test fields and the weight condition . . . 26
A.4 Transported (C1) . . . 27
A.5 Refutations and degeneracy of the encoding . . . 29
A.6 Analytic layer . . . 30
A.7 Periodic Clay statement . . . 34
A.8 Explicit exact solutions . . . 35

B Verification certificate **39**

1 Introduction and scope

Arnold's geometric picture of ideal hydrodynamics [1, 2] describes the Euler equations as Hamiltonian motion on coadjoint orbits of the group SDiff of volume-preserving diffeomorphisms, with Lie–Poisson bracket $\{F, G\}_{\mathrm{cl}}(\omega) = \int \omega \cdot (\delta F/\delta\omega \times \delta G/\delta\omega)$ and kinetic-energy Hamiltonian $H(\omega) = \frac{1}{2}\int |\mathrm{BS}\,\omega|^2$. The *Frohmanian tether* programme adds to this bracket a symmetric, negative-semidefinite quadratic term T and asks whether T can supply a damping mechanism for the vorticity sup-norm $M(t) = \|\omega(t)\|_{L^\infty}$, which by the Beale–Kato–Majda criterion [3] controls continuation of smooth Navier–Stokes solutions.

This paper reports exactly what the accompanying Lean 4 library proves. The Lean statements are the source of truth: every result below is stated to match a checked Lean declaration, with every hypothesis made explicit, and the exact Lean text is reproduced in Appendix A. The main findings are:

(i) The tether T satisfies (C3) unconditionally (Section 3). (C2) is not proved. (C1) with fixed functionals is refuted for T; the transported form of (C1) is proved for T under translations (Section 3.1). The identification $\delta H/\delta\omega = u$ assumed for (C2) in earlier versions is false with the library's definitions; this is proved in Lean (Theorem 3.6). The hypotheses formerly used for (C1) were false or unprovable and have been removed (Section 3.3).

(ii) *Obstruction to damping.* For every symmetric kernel B satisfying (C2), $B(H, M) = 0$ for every functional M; in particular, *if* T satisfied (C2), then $T(M, H) = T(H, M) = 0$ (Section 4). Either way, the symmetric design cannot both preserve the Euler vector field and produce the cubic damping rate on which the programme relied.

(iii) *Canonicity* holds only inside a restricted class and follows from saturation and polarization hypotheses; conditions (C1)–(C3) appear in the statement but are not used (Section 5). For T itself, nondegeneracy, polarization on differentiable functionals, saturation, projection-dependence and the canonical density are proved; unrestricted polarization and the pointwise weight hypothesis are refuted; the corrected a.e. weight condition is proved (Section 6).

(iv) The analytic layer consists of proved real-variable lemmas and *conditional* implications. Their remaining hypotheses are of two kinds (Section 11.1): classical theorems that are not in Mathlib (Kato local existence, Beale–Kato–Majda continuation, uniqueness), and hypotheses that are as strong as the open problem itself (a cubic rate or a uniform vorticity bound for all data) (Sections 8–9).

(v) *Explicit solutions* (Section 10). The decaying shear and ABC flows are proved to be nonzero exact solutions of the library's equations and, for $k = 2\pi m$, global smooth periodic solutions for their own data. No nonzero periodic field is a Kato witness (infinite energy on $\mathbb{R}^3$), so the $\mathbb{R}^3$ chain cannot be instantiated on them; this is proved, not assumed.

Scope statement. No result in this paper or in the library asserts global existence, smoothness or uniqueness of solutions of the three-dimensional Navier–Stokes equations. Every theorem whose conclusion mentions global solutions is an implication from hypotheses that are open. Section 11 lists precisely what the Lean development does and does not prove.

1.1 The results are conditional

Every theorem of this paper whose conclusion concerns Navier–Stokes solutions, and the canonicity theorems, is an *implication* whose hypotheses are not proved. The complete list of open hypotheses, by Lean name:

- *Classical results not in Mathlib:* `KatoLocalWitness` (argument `hKato`), `BKMGlobalContinuation`, `SmoothPhysicalUniqueness`, `PeriodicBKMContinuation`, `KatoUniquenessOnExistenceTimes`, `IsCoadjointMotion`.
- *As strong as the open problem:* `LocalTetherRate`, `LocalTetherBound`, `ClayStatements.LocalTetherClosingBound`, `ClayStatements.PeriodicTetherClosingBound`, `PeriodicDirectionDepletionBound`; global-path forms (auxiliary lemmas only) `FrohmanianTetherRate`, `FrohmanianTetherBound`, `FrohmanianTetherClosingBound`, `RiccatiMajorantExists`.
- *Kernel-class conditions on a competitor B* (canonicity theorems): symmetry, `PolarizesOnDifferentiable B`, `SaturatesTetherQuadratic B`, `h_proj`, `h_density`, `h_weight : AEEnstrophyWeight B`; and `InvariantUnderCoadjointAction`, `DegenerateWRTKineticEnergy`, `ProducesControllableNegativeFeedback` for B (carried, unused). All of them are proved for $B = T$ except (C2) (and except unrestricted polarization, which is refuted and not required).
- *For T itself:* `DegenerateWRTKineticEnergy TetherKernel` (C2) at regular vorticities, used as a hypothesis in Corollary 4.2; transport-invariance of T under rotations and general volume-preserving diffeomorphisms (not formalized).

None of these is an axiom; each is an explicit argument or a named `Prop`.

Version note. Version 0.1.0 of this manuscript named a different repository (`NS_Millennium_Proof`), quoted theorem and job counts of that repository, and cited a Zenodo DOI belonging to it. Version 0.1.1 refers only to the standalone repository above and uses the counts produced by its own build (Appendix B). Version 0.1.2 removes the hypothesis `KineticEnergyDerivIsVelocity` ($\delta H/\delta\omega = u$) and every theorem that assumed it or its pointwise form (the (C2) theorems `tetherKernel_degenerates_on_kinetic_energy`, `tetherKernel_C2_of_identifications` and `Combined.tetherKernel_C2_of_KEDeriv`; `tethered_reproduces_classical_euler`; `degeneracy_for_mollified_sup_norm_proxy`; the first clause of `frohmanian_tether_theorem`; the (C2) and no-damping conclusions of the combined capstone), together with the lemmas `functional_derivative_eq_velocity_of_unique_repr` and its two aliases, whose hypotheses can never hold (Theorem 3.7). It adds the Lean refutation of the hypothesis (Section 3.2). It further removes the false or vacuous hypotheses listed in Section 3.3, together with every theorem that depended on them, and repairs `IsPhysicalNSSolution` (Section 2). A third pass replaces the choice-based functional derivative by the unique L^2 gradient (Section 2), settles several kernel conditions for T (Section 5), and restricts the Kato witness to its interval (Section 8). A fourth pass refutes the unrestricted weight hypothesis and the fixed-functional (C1) for T; a fifth pass constructs concentrated divergence-free test fields, proves the corrected weight condition for T (Section 6) and proves the transported (C1) for

translations (Section 3.1). Version 0.1.3 (10.5281/zenodo.24102715) added only its DOI to 0.1.2 (10.5281/zenodo.24102345). This version, 0.1.4 (DOI (to be assigned on publication)), adds the module `ExplicitFlows` (Section 10): the decaying shear and ABC flows as exact solutions, their periodic smoothness, and the proof that no nonzero periodic field can be a `KatoLocalWitness`. No existing Lean statement was changed. All versions are collected under the concept DOI 10.5281/zenodo.24081999. The Zenodo version DOI of this repository's `v0.1.1` archive is 10.5281/zenodo.24082000 (GitHub tag `v0.1.1` / commit `0722b16`); it still contains the removed hypothesis `KineticEnergyDerivIsVelocity`. The older-project DOI `10.5281/zenodo.23717084` belongs to `NS_Millennium_Proof` and does *not* identify this repository. This manuscript is not peer-reviewed and is not an unconditional Navier–Stokes proof.

2 The formal setting

We record the definitions as formalized, including the points at which they differ from the classical objects.

- **Space.** `T3` is `EuclideanSpace ℝ (Fin 3)`, i.e. $\mathbb{R}^3$ with Lebesgue measure, *not* the quotient torus $\mathbb{R}^3/\mathbb{Z}^3$. Periodicity, where needed (Section 9), is the predicate `IsZ3Periodic` on functions on $\mathbb{R}^3$. The notation SDiff($\mathbb{T}^3$) in the title refers to the motivating geometry; no diffeomorphism group of $\mathbb{T}^3$ is constructed.

- **Phase space.** `CoadjointOrbit` is the type of pointwise divergence-free vector fields $\omega : \mathbb{R}^3 \to \mathbb{R}^3$ (a subtype; no orbit structure is built in).

- **Functional derivative.** Let L^2 be `Lp (EuclideanSpace ℝ (Fin 3)) 2 volume`. A *test direction* (`IsTestDirection`) is a C^∞, compactly supported field in `CoadjointOrbit` (hence divergence-free); `DivFreeL2` is the closure in L^2 of the span of the test directions. A vorticity is *regular* (`IsRegularVorticity`) if each component is differentiable everywhere. `IsFunctionalGradient F ω g` says: $g \in$ `DivFreeL2`, ω is regular, and $\frac{d}{dt}\big|_{t=0} F(\omega + t\varepsilon) = \langle g, \varepsilon\rangle_{L^2}$ for every test direction ε. Such a g is unique (`isFunctionalGradient_unique`: the difference of two gradients is orthogonal to a dense subset of `DivFreeL2`). `FunctionalDerivative F ω` is (a representative of) this unique g when it exists, and 0 otherwise; in particular it is 0 at every non-regular ω (`functionalDerivative_eq_zero_of_not_regular`). The value is determined up to a null set, and no arbitrary choice among different derivatives is made. Earlier versions chose a Gâteaux representative, which is never unique (Theorem 3.7); the predicate `IsGateauxRepresentative` is kept only for that record.

- **Velocity and energy.** `velocity_from_vorticity ω` is the Biot–Savart integral `BiotSavart ω.val`, and `KineticEnergyHamiltonian ω` $= \frac{1}{2}\int \|\mathrm{BS}\,\omega\|^2$.

- **Navier–Stokes.** `navier_stokes_eq u p ν` (and `NS_PDE`) asserts the momentum equation with Mathlib's total derivative `deriv` in time, and $\nabla \cdot u = 0$, pointwise for $t \geq 0$. `IsPhysicalNSSolution u` adds pointwise time-differentiability for $t \geq 0$, spatial continuity, finite kinetic energy, the energy inequality $\int \|u(t)\|^2 \leq \int \|u(0)\|^2$, and an *affine extension to negative times*: there is a field v_0 with $u(t) = u(0) + t\,v_0$ for all $t \leq 0$.

Repair of `IsPhysicalNSSolution`. Earlier versions required $u(t) = u(0)$ for $t \leq 0$. Since `deriv` is the two-sided derivative, differentiability at $t = 0$ then forced $\partial_t u(0) = 0$, so a solution whose velocity changes at the initial time could never qualify; with the energy inequality, this made the class essentially one of solutions that are stationary at $t = 0$. The negative-time clause exists only to stop junk values at $t < 0$ from interacting with `deriv`

at 0; the affine extension does this and allows any initial time derivative v_0. The sanity lemma `PhysicalSolutionSanity.exists_physical_time_deriv_ne_zero` exhibits a path u satisfying `IsPhysicalNSSolution` whose time derivative at $t = 0$ is nonzero at some point (`not_forall_physical_time_deriv_zero` states the same as the failure of the old consequence). The path is $g(t)\,\varphi$ with $\varphi(x) = \max(0, 1 - \|x\|)\,e_0$, $g(t) = 1 - t$ for $t \le 0$ and $g(t) = 1/(1+t)$ for $t > 0$; it is *not* a Navier–Stokes solution. The library's nonzero Navier–Stokes solutions (the constant flows and the shear and ABC flows of Section 10) all have infinite energy on $\mathbb{R}^3$ and so are not in this class; non-vacuity of the class for genuine non-steady flows is not formalized. The exact definitions are quoted in Appendix A.1.

3 The tether kernel and conditions (C1)–(C3)

Definition 3.1 (`Pi_u`, `TetherKernel`, `TetheredBracket`)**.** For velocity fields u, v put $\Pi_u v = v - \big(\int \langle v, u\rangle / \int \|u\|^2\big)\,u$ (in Lean, division by 0 is 0, so $\Pi_u v = v$ when $\int \|u\|^2 = 0$). With $\kappa = 3/2$ (`kappa_eq_three_halves`), $\mathcal{E}(\omega) = \int \|\omega\|^2$ (`Enstrophy`) and $u = \mathrm{BS}\,\omega$,

$$T(\omega; F, G) = -\kappa\,\mathcal{E}(\omega) \int \big\langle \Pi_u\,\delta F/\delta\omega,\ \Pi_u\,\delta G/\delta\omega \big\rangle\,dx, \qquad \{F, G\}_{\mathfrak{T}} = \{F, G\}_{\mathrm{cl}} + T(F, G).$$

The three conditions are predicates on a kernel $B(\omega; F, G)$: (C1) `InvariantUnderCoadjointAction`: $B(g\cdot\omega; F, G) = B(\omega; F, G)$ for all g in a group `SDiff` acting on `CoadjointOrbit`; (C2) `DegenerateWRTKineticEnergy`: $B(\omega; F, H) = 0$ for all F, ω; (C3) `ProducesControllableNegativeFeedback`: $B(\omega; F, F) \le -\kappa\,\mathcal{E}(\omega) \int \|\Pi_u\,\delta F/\delta\omega\|^2$.

Theorem 3.2 (Symmetry and (C3); `tetherKernel_comm`, `tetherKernel_C3`)**.** *For all ω, F, G: $T(\omega; F, G) = T(\omega; G, F)$, and T satisfies (C3). No hypotheses.*

(C2) is not proved. No theorem of the library asserts that T satisfies (C2). The only (C2) fact proved for T is the degenerate case `tetherKernel_C2_of_no_gateaux`: if H has no functional gradient at ω (for example, ω not regular), then $T(\omega; F, H) = 0$ for every F (because the derivative is then 0). At regular ω, (C2) would need $\delta H/\delta\omega$, classically the Leray part of the stream function; the Biot–Savart operator is unbounded on $L^2(\mathbb{R}^3)$, this is not in Mathlib, and (C2) is expected to fail there (see below). Section 3.2 explains why the former route to (C2) was removed.

(C1) with fixed functionals is refuted for T under the concrete action. Earlier versions derived (C1) for T from two hypotheses, `SDiffActionHyp` with a transitivity field and `TetherIntegralPreserved`; both are removed (Section 3.3) and no (C1) theorem for T remains. `SDiffActionHyp` now has the single field `casimir_invariant` (every Casimir $\int f(|\omega|)$ is invariant under the action). For the concrete group `ConcreteSDiff.SDiffConcrete` of measure-preserving, measurable, divergence-preserving permutations of $\mathbb{R}^3$ acting by $g \cdot \omega = \omega \circ g^{-1}$ (scalar composition, not the geometric push-forward $(Dg\,\omega) \circ g^{-1}$), the structure is proved outright (`ConcreteSDiff.sdiffActionHyp`).

Theorem 3.3 (`not_tetherKernel_invariant_concrete`)**.** *T does not satisfy (C1) for the concrete group:* `¬ InvariantUnderCoadjointAction ConcreteSDiff.G TetherKernel`.

The translations $x \mapsto x + h$ belong to the concrete group (`ConcreteSDiff.translate`). Take g the translation by $3e_0$, $\omega = \rho_0$ and $g \cdot \omega = \rho_{3e_0}$ (rotation fields, see Section 5), and a rotation field $\varphi \in \{\rho_0, \rho_{3e_0}\}$ with $T(\omega; F_\varphi, F_\varphi) \neq 0$ (`tetherKernel_rot_nondegenerate`). The functional $F(\eta) = |F_\varphi(\eta) - F_\varphi(g\cdot\omega)|$ has no gradient at $g \cdot \omega$, so $T(g\cdot\omega; F, F) = 0$; since $F_\varphi(\omega) \neq F_\varphi(g\cdot\omega)$, at ω it has gradient $\pm\varphi$ and $T(\omega; F, F) = T(\omega; F_\varphi, F_\varphi) \neq 0$. The predicate `InvariantUnderCoadjointAction` keeps F, G fixed while ω moves, which is not the physically

meaningful notion: the transported form, with F, G moved along with ω, is proved for translations in Section 3.1.

3.1 Transported (equivariant) (C1)

Definition 3.4 (`InvariantUnderTransportedCoadjointAction`)**.** For a group S acting on `CoadjointOrbit`, a kernel B is *transport-invariant* if $B(g \cdot \omega;\, g_*F,\, g_*G) = B(\omega; F, G)$ for all g, F, G, ω, where $(g_*F)(\eta) = F(g^{-1} \cdot \eta)$.

Theorem 3.5 (`tetherKernel_transported_invariant`)**.** *T is transport-invariant for the translation subgroup `ConcreteSDiff.transSubgroup` of the concrete group (the elements `translate h`, $x \mapsto x + h$, with $(\tau_h \cdot \omega)(x) = \omega(x - h)$, `translate_smul_val`).*

Proof (formalized). Let L_h be translation on L^2, $L_h v = v(\cdot - h)$; in Lean it is Mathlib's `Lp.compMeasurePreservingₗᵢ` along the measure-preserving map $x \mapsto x - h$ (`measurePreserving_sub_right`), a linear isometry. (1) L_h maps each test field to a test field and so, being continuous, maps `DivFreeL2` (closure of their span) into itself (`l2Translate_mem_divFreeL2`, Mathlib `map_mem_closure`). (2) If g is the functional gradient of F at ω, then $L_h g$ is the gradient of $\tau_{h*}F$ at $\tau_h \cdot \omega$ (`isFunctionalGradient_translate`): the line $\tau_h^{-1} \cdot (\tau_h \omega + t\varepsilon)$ is $\omega + t\,\tau_h^{-1}\varepsilon$, and $\langle g, \tau_h^{-1}\varepsilon\rangle = \langle L_h g, \varepsilon\rangle$ (`LinearIsometry.inner_map_map`). Applying this to $-h$ gives the converse, so $\delta(\tau_{h*}F)/\delta\omega(\tau_h\omega) = (\delta F/\delta\omega(\omega))(\cdot - h)$ a.e. in all cases (`functionalDerivative_translate`). (3) The Biot–Savart integral commutes with translations, $\mathrm{BS}(\tau_h\omega) = (\mathrm{BS}\,\omega)(\cdot - h)$ (`biotSavart_translate`; Mathlib `integral_add_right_eq_self`), hence so do the Gram–Schmidt coefficient of Π_u and the projected pairing (`tI_translate`, Mathlib `integral_sub_right_eq_self`). (4) The enstrophy is invariant (`ConcreteSDiff.enstrophy_invariant`).

Sanity. The property is not vacuous: the point-evaluation kernel $B(\omega; F, G) = |\omega(e_0/2)|$ fails it (`not_transported_invariant_pointEval`). It differs from the fixed-functional (C1): on the same translation subgroup the latter fails for T (`not_tetherKernel_invariant_transSubgroup`, via the generic `not_tetherKernel_invariant_of_shift`), while the transported form holds.

Open: rotations and general diffeomorphisms. The concrete group acts by scalar composition $\omega \circ g^{-1}$, which is the correct transport of a vorticity only for translations; for a rotation R the geometric action is $R\,\omega(R^{-1}x)$ and for a volume-preserving diffeomorphism φ it is the push-forward $(D\varphi\,\omega) \circ \varphi^{-1}$. Neither action is formalized, and transport-invariance of T under them is open. Under the full concrete group (which contains non-smooth measure-preserving maps) one expects transport-invariance to fail, since such maps destroy regularity and hence gradients.

3.2 The identification $\delta H/\delta\omega = u$ is false (removed)

Earlier versions used the named proposition `PlanSteps.KineticEnergyDerivIsVelocity`: *for every ω, `FunctionalDerivative KineticEnergyHamiltonian ω = velocity_from_vorticity ω`*, as the hypothesis of (C2) for T. It has been deleted from the library, together with every theorem that assumed it or its pointwise form, because it is false.

Theorem 3.6 (Refutation of $\delta H/\delta\omega = u$)**.** *(Lean: `KineticEnergyDerivative.not_forall_functionalDerivative_kineticEnergy_eq_velocity`.) It is not the case that `FunctionalDerivative KineticEnergyHamiltonian ω = velocity_from_vorticity ω` for every `ω : CoadjointOrbit`.*

Proof idea (formalized). Let B be the open unit ball centred at $(0, 2, 0)$ and $\omega = \mathbf{1}_B\, e_1$. Each coordinate of ω takes two values, so its Fréchet derivative is 0 everywhere (Mathlib's `fderiv` is 0 at non-differentiable points, and a two-valued function differentiable at a point is locally constant there); hence $\nabla \cdot \omega = 0$ in the library's sense and ω is a point of `CoadjointOrbit`. It is not regular (the first coordinate jumps at $(1, 2, 0)$), so `FunctionalDerivative` is 0 at ω. But the third component of $\mathrm{BS}\,\omega$ at the origin is $-(4\pi)^{-1} \int_B y_2\, |y|^{-3}\, dy < 0$, so $\mathrm{BS}\,\omega \neq 0$. (Under the former choice-based encoding the argument compared ω with 2ω.) Theorem 3.6 uses only the three standard axioms.

Theorem 3.7 (Non-uniqueness of Gâteaux representatives)**.** *(Lean:* `KineticEnergyDerivative.not_unique_isGateauxRepresentative`*,* `KineticEnergyDerivative.not_unique_velocity_representative`*.) If δ is a Gâteaux representative of F at ω, so is a field $\delta' \neq \delta$ (change δ at the origin). Consequently, for no ω is* $\mathrm{BS}\,\omega$ *the* unique *Gâteaux representative of H at ω.*

Scope of the refutation. The counterexample is a non-smooth vorticity field, at which the derivative is 0 by definition; it shows that the Lean hypothesis, as stated, is false, and therefore that every theorem which assumed it was vacuous. It does not by itself settle what the derivative is at smooth fields. For the L^2 vorticity pairing used by `IsFunctionalGradient`, the classical formal computation gives $\delta H/\delta\omega = \mathrm{BS}^*\mathrm{BS}\,\omega$, the vector stream function ψ with $\nabla \times \psi = u$, not u; this identity is *not* formalized. What is formalized is the velocity-pairing statement `isVelocityGateauxRepresentative_kineticEnergy_at`: with the pairing $\int \langle \delta, \mathrm{BS}\,\varepsilon \rangle$, the derivative of H along ε is represented by u (under explicit integrability hypotheses). Whether T, with ψ in place of u, satisfies (C2) is open and in general expected to fail: $\Pi_u \psi = 0$ would require ψ to be parallel to u.

3.3 Other removed hypotheses (version 0.1.2)

Each item below was an explicit hypothesis of earlier versions. Each was false, unprovable in the encoding, or vacuous, and has been deleted together with every theorem that used it. No remaining statement was weakened silently: a theorem whose proof needed a removed hypothesis was deleted, not re-proved under a weaker one. All refutations use only the three standard axioms.

1. **Transitivity on Casimir levels** (field `transitive_on_casimir_levels` of `SDiffActionHyp`: fields with equal Casimirs lie in one orbit). *Refuted* for the concrete action: `ConcreteSDiff.not_transitive_on_casimir_levels`. The constant fields e_0 and e_1 have the same Casimirs but different orbits, because the action fixes every constant field. Deleted with it: `orbit_eq_casimir_level`.

2. **`TetherIntegralPreserved`** (invariance of $\int \|\Pi_u\, \delta F/\delta\omega\|^2$ under the action). It concerned the unconstrained choice made by the former `FunctionalDerivative`, so it could be neither proved nor refuted from the definitions. It was not reinstated after the rebuild. Deleted with it: `tether_coadjoint_invariance`, `tether_satisfies_C1`, `tetherKernel_C1`, `Combined.tetherKernel_C1_of_casimir`, `Combined.tetherKernel_C1_of_volumePreserving` (whose structure field carried the same assumption), `Combined.combined_conditional_capstone`, and related bridges.

3. **Pointwise strain bound** `PointwiseStrainSupBound`: $\|\nabla u(x)\| \leq C\|\omega\|_{L^\infty}$. *Refuted*: `StrainBoundRefutation.not_forall_strain_le_sup_vorticity`. For every C, the linear strain field $u(y) = (y_1, y_0, 0)$ has $\nabla \times u = 0$ and $\nabla u \neq 0$. (Classically the bound fails even for smooth decaying fields, since the Calderón–Zygmund operator is

unbounded on L^∞.) Deleted with it: the stretching estimates at a spatial maximum (`stretching_inner_le_at_max` and relatives), `tethered_stretching_of_weight`, `reduced_pairing_of_unmodified_and_tether` and the other conditional bridges built on them.

4. **Circular Jacobi lemma.** `tethered_jacobi_identity` assumed that the Jacobiator equals a cyclic integral (`hident`) and that this integral vanishes (`hvanish`), i.e. its conclusion. Deleted. No Jacobi identity is claimed for $\{\cdot,\cdot\}_{\mathfrak{T}}$.

5. **Antisymmetric uniqueness clause.** The former first clause of `frohmanian_tether_theorem` (and `uniqueness_of_minimal_tether_antisymmetric`) quantified over antisymmetric kernels that saturate T. Proved: `antisymmetric_clause_forces_zero`. Antisymmetry, saturation and the two polarization hypotheses force $B = 0$ *and* $T = 0$, so the clause applied to no kernel unless $T \equiv 0$. Deleted.

6. **Uniform lower bound `hTlb` on local existence times**, with the restart hypotheses `hInt`, `huniq_shift` of `frohmanian_tether_theorem`. A data-independent lower bound on Kato existence times for all smooth finite-energy data is not known. By Navier–Stokes scaling it is of the same strength as the open problem; it is not known to be false. It was removed and the theorem re-proved through the BKM route (Theorem 8.6).

7. **Unrestricted polarization of T** (`Polarizes TetherKernel`). *Refuted*: `TetherNondegeneracy.not_tetherKernel_polarizes`. The functional $F(\omega) = |\int\langle\omega,\varphi\rangle|$ (`absFun`) has no gradient at $\omega = 0$, so its derivative is 0 there, while $F + G$ for a suitable linear G has one; the identity fails. It is replaced by `PolarizesOnDifferentiable` (polarization for functionals that have gradients), which is *proved* for T (`tetherKernel_polarizesOnDifferentiable`).

8. **Pointwise weight hypothesis** `h_weight` (every density equals $-c\,\mathcal{E}(\omega)$ at *every* x). *Refuted* for every kernel that has a density (`not_pointwiseEnstrophyWeight_of_density`): changing a density at $x = 0$ gives another density, since densities enter only through integrals. The former core theorem that assumed it was therefore vacuous. It is replaced by the almost-everywhere form `AEEnstrophyWeight` (on regular ω).

9. **Almost-everywhere weight hypothesis over all densities** (`AEEnstrophyWeightUnrestricted`: every density, measurable or not, equals $-c\,\mathcal{E}(\omega)$ a.e. at regular ω). *Refuted* for T: `TetherWeight.not_aeEnstrophyWeightUnrestricted_tetherKernel`. Let $V \subset B(0,1)$ contain one point of each class of $\mathbb{R}^3/\mathbb{Q}e_0$ meeting the unit ball (`vitaliSet`). Its disjoint rational translates show that every null-measurable subset of V is null, and they cover $B(0,1)$, so V is not null. Hence $\int 1_V f = 0$ for every f (`integral_indicator_vitaliSet_mul`): either $1_V f$ is not integrable, or its support is a null-measurable subset of V. At the regular vorticity $\omega = 0$ we have $\mathcal{E}(0) = 0$, so $T(0;\cdot,\cdot) = 0$ and 1_V is a density of T there, but $1_V \neq 0$ on a non-null set. Every kernel equal to T therefore fails this hypothesis (`not_aeEnstrophyWeightUnrestricted_of_eq_tether`), so the core theorem stated with it had jointly unsatisfiable hypotheses. `AEEnstrophyWeight` now quantifies only over densities that are locally integrable at every regular ω, and `h_density` asks for such a density; T has one (`tetherKernel_has_locallyIntegrable_density`).

10. **`Step1Hyp`** (derivative of a difference is the difference of derivatives, for all functionals). *Refuted*: `not_step1Hyp`, via `TetherNondegeneracy.not_forall_functionalDerivative_sub`. Deleted with it: `step1_locality_sym`, `step1_locality_route1`, `tetherKernel_polarizes`.

11. **Global equations in the Kato witness.** The field `pde : NS_PDE u p ν` of `KatoLocalWitness` asked for the equations at every $t \geq 0$, which is stronger than Kato's local theorem. It is now `pde : NS_PDE_On u p ν T` (equations on $[0, T]$) with `phys : IsPhysicalNSSolutionOn u T`. Deleted because they used the global field: the restart concatenation `mem_existenceTimes_add_of_kato_restart` (which would need a gluing statement), `conditional_regularity_of_bkmIntegrability`, `conditional_regularity_scaffold`, `conditional_regularity_of_restart`, `conditional_regularity_of_bkm_uniqueness` and `conditional_regularity_of_kato_estimates`. The hypotheses `hcrit` (BKM criterion via `Tstar`) and the global-path rate in `frohmanian_tether_theorem` are replaced by the explicit continuation hypothesis `BKMGlobalContinuation` and the local-solution rate `LocalTetherRate` (Section 8).

The encoding at non-regular vorticities. At a vorticity with a non-differentiable component, `FunctionalDerivative F ω` is 0 for every F (`functionalDerivative_indep_of_not_differentiable`), so $T(\omega; F, G) = 0$ there (`tetherKernel_indep_of_not_differentiable`). This is now a property of a genuine derivative (no derivative exists there), not an artefact of an unspecified choice. The record lemma `polarizes_tetherKernel_zero_of_not_differentiable` remains true but its hypothesis is refuted.

4 The obstruction to damping

Theorem 4.1 (`Combined.left_H_zero_of_C2_symm`)**.** *Let B be any kernel that is symmetric, $B(\omega; F, G) = B(\omega; G, F)$, and satisfies (C2). Then $B(\omega; H, M) = 0$ for every ω and every functional M.*

Corollary 4.2 (`Combined.tetherKernel_left_H_zero`)**.** *If T satisfies (C2), then $T(\omega; H, M) = T(\omega; M, H) = 0$ for every ω and M.*

Theorem 4.1 is unconditional. For the tether, Corollary 4.2 requires (C2) as a hypothesis, and (C2) for T is not proved (Section 3.2). The interpretation, for T or any symmetric kernel that satisfies (C2), is the following. Along the tethered flow $\dot{M} = \{M, H\}_{\mathfrak{T}} = \{M, H\}_{\mathrm{cl}} + T(M, H)$, the tether term would vanish; hence the symmetric design that would make the tether preserve the Euler vector field (C2) would also force its contribution to the evolution of every functional, including smoothed versions of $\|\omega\|_{L^\infty}$, to be zero. The negative diagonal values $T(F, F) \leq 0$ of (C3) never enter dynamics generated by H. Consequently the cubic rate $M' \leq CM^2 - \kappa'' M^3$ used below is not a consequence of the tether: under (C2) its damping coefficient would be 0, and without (C2) nothing about it is proved.

A related formal fact: `PlanSteps.tether_bound_not_scale_invariant`: for $\kappa > 0$, $M \neq 0$ and any real S, the family of inequalities $\lambda^6 S \leq C(\lambda^2 M)^3 - \kappa(\lambda^2 M)^4$ $(\lambda > 0)$ fails for some λ, so a pointwise bound of this shape cannot be invariant under Navier–Stokes scaling.

5 Canonicity

Theorem 5.1 (`uniqueness_of_minimal_tether`; delegates to `uniqueness_of_minimal_symmetric_tether`)**.** *Let a group act on `CoadjointOrbit` and assume `SDiffActionHyp` (proved for the concrete group). Let B be symmetric, satisfy (C1), (C2), (C3), `SaturatesTetherQuadratic B` (i.e. $B(\omega; F, F) = T(\omega; F, F)$ for all F, ω) and `PolarizesOnDifferentiable B` (the polarization identity $2B(F, G) = B(F+G, F+G) - B(F, F) - B(G, G)$ whenever F and G have functional gradients at ω). Then $B(\omega; F, G) = T(\omega; F, G)$ whenever F and G have functional gradients at ω.*

Remark 5.2. The proof uses only saturation, polarization of B, and polarization of T on functionals with gradients, which is now *proved* (`tetherKernel_polarizesOnDifferentiable`). The hypotheses `SDiffActionHyp` and (C1)–(C3) are not used. Theorem 5.1 is therefore a uniqueness statement *given* agreement on the diagonal, not a derivation of T from (C1)–(C3). Unrestricted polarization of T is false (Section 3.3).

Theorem 5.3 (`uniqueness_of_minimal_tether_core`)**.** *Let B be symmetric and assume: $B(\omega; F, G)$ depends on F only through $\Pi_u\,\delta F/\delta\omega$ (`h_proj`); if B factors through the projection then it has a density α (`HasTetherKernelDensity`) with $\alpha(\omega,\cdot)$ locally integrable at every regular ω (`h_density`); every such locally integrable density satisfies, for some c and every regular ω, $\alpha(\omega, x) = -c\,\mathcal{E}(\omega)$ for almost every x (`h_weight : AEEnstrophyWeight B`); and B saturates T on the diagonal (`h_sat`). Then $B = T$.*

The former nondegeneracy hypothesis `h_nondeg` is no longer needed and is *proved* (`tether_nondeg_witness`, `TetherNondegeneracy.tetherKernel_nondegenerate`): there are a regular ω and a smooth test field φ with $T(\omega; F_\varphi, F_\varphi) \neq 0$ for the linear functional $F_\varphi(\omega) = \int\langle\omega, \varphi\rangle$. The witness uses rotation fields $\rho_c(y) = \chi(1 - |y - c|^2)\,R(y - c)$ (χ a smooth cut-off, R a rotation generator) with disjoint supports at $c = 0$ and $c = 3e_0$, so that $\Pi_u\varphi \neq 0$.

Status of the kernel conditions for T.

Condition	Status	Lean / reason
nondegeneracy (`h_nondeg`)	proved	`tetherKernel_nondegenerate`
`Polarizes TetherKernel`	refuted	`not_tetherKernel_polarizes`
`PolarizesOnDifferentiable TetherKernel`	proved	`tetherKernel_polarizesOnDifferentiable`
`SaturatesTetherQuadratic TetherKernel`	proved	`tetherKernel_kernel_conditions`
`h_proj` for T	proved	`tetherKernel_kernel_conditions`
`h_density` for T	proved	canonical density (locally integrable), `tetherKernel_kernel_conditions`, `tetherKernel_has_locallyIntegrable_density`
pointwise `h_weight`	refuted	for every kernel with a density, `not_pointwiseEnstrophyWeight_of_density`
a.e. `h_weight` over all densities	refuted	Vitali-set density at $\omega = 0$, `not_aeEnstrophyWeight UnrestrictedTetherKernel`
`AEEnstrophyWeight TetherKernel` (locally integrable densities)	proved	`tetherKernel_aeEnstrophyWeight` (Section 6)
`Step1Hyp`	refuted	`not_step1Hyp`
(C2) for T	open	needs $\delta H/\delta\omega$ at regular ω; expected to fail
(C1) for T (concrete action, F, G fixed)	refuted	`not_tetherKernel_invariant_concrete` (translation)
transported (C1) for T, translations	proved	`tetherKernel_transported_invariant`
transported (C1) for T, rotations / diffeomorphisms	open	geometric push-forward action not formalized

The conditions `PolarizesOnDifferentiable B`, `SaturatesTetherQuadratic B`, `h_proj`, `h_density`, `h_weight` for a *general* competitor B remain hypotheses: they define the class of kernels compared with T.

6 Concentrated divergence-free test fields and the weight condition

The family. For a smooth compactly supported scalar H put $\operatorname{curl}(He_2) = (\partial_1 H, -\partial_0 H, 0)$ (`curlVal`, with $\partial_i H(y)$ Mathlib's `fderiv ℝ H y (EuclideanSpace.single i 1)`). It is smooth, has compact support inside `tsupport H`, and is divergence-free (`div_curlVal`, by symmetry of second derivatives, Mathlib `ContDiffAt.isSymmSndFDerivAt`); hence it is a test direction (`curlOrbit_test`). With H a Mathlib `ContDiffBump` centred at x (1 on $B(x, r/4)$, 0 outside $B(x, r/2)$), `concField x hr` is a test field supported in $B(x, r)$ (`tsupport_concField_subset`) and not identically zero (`concField_ne_zero`), for every point x and every radius $r > 0$.

Orthogonality to u. The tether uses $\Pi_u v = v - \lambda(v)u$ with $\lambda(v) = \int\langle v, u\rangle / \int \|u\|^2$ (`projCoef`), and $u = \mathrm{BS}\,\omega$ is not compactly supported, so the concentrated fields are not themselves orthogonal to u. The proof avoids constructing orthogonal fields: λ is additive on test fields (`projCoef_add_smul`), and two auxiliary test fields with $\lambda \neq 0$, supported away from the support of a, b, can be subtracted from a and b to make them orthogonal to u without changing $\langle a, b\rangle$ pointwise (`good_of_two_correctors`). This works near every point except at most two (`bad_finite`), a null set.

Theorem 6.1 (`tetherKernel_aeEnstrophyWeight`)**.** *`AEEnstrophyWeight TetherKernel` holds with $c = \kappa$: every density α of T with $\alpha(\omega, \cdot)$ locally integrable at every regular ω satisfies $\alpha(\omega, x) = -\kappa\,\mathcal{E}(\omega)$ for almost every x, at every regular ω.*

Proof (formalized). Put $\beta = \alpha(\omega, \cdot) + \kappa\mathcal{E}(\omega)$ (locally integrable). For test fields a, b with $\lambda(a) = \lambda(b) = 0$, the gradients of the linear functionals F_a, F_b are a, b, $\Pi_u a = a$, $\Pi_u b = b$ a.e., and the density equation gives $\int \beta\langle a, b\rangle = 0$ (`integral_beta_eq_zero`); all products are integrable (Mathlib `LocallyIntegrable.integrable_smul_right_of_hasCompactSupport`). With the correctors this holds for all test pairs supported in a small ball around any good point. On such a ball, for every smooth g with compact support in it and a bump $\chi \equiv 1$ near $\operatorname{supp} g$, $g = \langle \operatorname{curl}(x_0 g\, e_2), \operatorname{curl}(x_0 \chi\, e_2)\rangle - \langle \operatorname{curl}(g\, e_2), \operatorname{curl}(\tfrac{1}{2}x_0^2 \chi\, e_2)\rangle$ pointwise (`pointwise_identity`), so $\int \beta g = 0$, and Mathlib's `IsOpen.ae_eq_zero_of_integral_contDiff_smul_eq_zero` gives $\beta = 0$ a.e. on the ball (`ae_zero_on_good_ball`). A countable subcover (Mathlib `TopologicalSpace.isOpen_iUnion_countable`) and the finite bad set give $\beta = 0$ a.e. No Lebesgue-differentiation step is needed beyond Mathlib's fundamental lemma of the calculus of variations.

Sanity. The hypotheses are satisfiable and the conclusion is not vacuous (`aeEnstrophyWeight_nontrivial`): the canonical density is a locally integrable density of T, and at the regular vorticity ρ_0 the weight $-\kappa\mathcal{E}(\rho_0)$ is nonzero. The unrestricted form over all densities is false (Section 3.3). Consequently the density and weight inputs of `uniqueness_of_minimal_tether_core` hold for $B = T$ (`tetherKernel_core_inputs`); for a general competitor B they remain hypotheses.

7 Bracket structure and Casimirs

- `classicalBracket_antisymm`: $\{F, G\}_{\mathrm{cl}} = -\{G, F\}_{\mathrm{cl}}$; `classicalBracket_H_H`: $\{H, H\}_{\mathrm{cl}} = 0$. The Jacobi identity of $\{\cdot,\cdot\}_{\mathrm{cl}}$ is classical [1] and is *not* formalized.
- Since T is symmetric and in general nonzero, $\{\cdot,\cdot\}_{\mathfrak{T}}$ is *not* antisymmetric; it is of metriplectic type in the sense of Morrison [7] (antisymmetric part plus symmetric negative-semidefinite part). No Jacobi identity is claimed for it.

- `PlanSteps.casimir_conserved`: under `SDiffActionHyp` and `IsCoadjointMotion` (the trajectory is $\omega(t) = g(t) \cdot \omega(0)$), every Casimir is constant along the trajectory. That the Euler flow is such a motion is Arnold's classical picture but is not formalized (Section 11.1).

- `ScalingIdentities.vorticity_rescale`: for $u_\lambda(x) = \lambda u(\lambda x)$, $\nabla \times u_\lambda(x) = \lambda^2(\nabla \times u)(\lambda x)$ under differentiability at λx.

8 The analytic layer

Lemma 8.1 (Comparison; `AnalyticPipeline.comparison_ode_stability`)**.** *Let $C, \kappa'' > 0$ and let $y : \mathbb{R} \to \mathbb{R}$ be differentiable at every $s \geq 0$ with $y'(s) \leq Cy(s)^2 - \kappa'' y(s)^3$ for $s \geq 0$. Then $y(t) \leq \max(y(0), C/\kappa'')$ for every $t \geq 0$.*

Lemma 8.2 (Finite-interval comparison; `TetheredLyapunov.comparison_ode_stability_Icc`)**.** *The same conclusion holds on $[0, T]$: if y is differentiable on $[0, T]$ with $y' \leq Cy^2 - \kappa'' y^3$ there, then $y(t) \leq \max(y(0), C/\kappa'')$ for $t \in [0, T]$.*

Proposition 8.3 (`riccati_ceiling_of_vorticity_di`)**.** *If $t \mapsto M(t) = \sup_x \|(\nabla \times u(t))(x)\|$ is continuous, differentiable at every $s \geq 0$, and satisfies $M'(s) \leq CM(s)^2 - \kappa'' M(s)^3$ for $s \geq 0$ with $C, \kappa'' > 0$, then M is bounded on $[0, \infty)$.*

Local solutions. `NS_PDE_On u p ν T` asks for the equations for $t \in [0, T]$ only, and `IsPhysicalNSSolutionOn u T` is the physical-solution predicate on $[0, T]$. `IsLocalPhysicalSolution ν T u₀ u p` combines $u(0) = u_0$, `NS_PDE_On`, smoothness on $[0, T]$ and `IsPhysicalNSSolutionOn`.

The following hypotheses are named propositions (exact text in Appendix A.6); all are open for general data or are classical theorems that are not formalized:

- `KatoLocalWitness u₀ ν`: a time $T > 0$ and (u, p) with $u(0) = u_0$, smooth on $[0, T]$, satisfying `NS_PDE_On u p ν T` and `IsPhysicalNSSolutionOn u T`. This is the conclusion of Kato's local theorem [6] for the library's data class; it is not formalized, and a witness is an input (hKato).

- `LocalTetherRate u₀ ν` (hDI): there are $C, \kappa'' > 0$, fixed before the interval, such that every local physical solution from u_0 on every $[0, T]$ has M differentiable on $[0, T]$ with $M' \leq CM^2 - \kappa'' M^3$ there. (Differentiability of a sup-norm in time is itself a strong requirement.) By the finite-interval comparison lemma it implies `LocalTetherBound u₀ ν` (`localTetherBound_of_localTetherRate`).

- `LocalTetherBound u₀ ν`: one ceiling Y with $M(t) \leq Y$ for every local physical solution from u_0 on every $[0, T]$ and every $t \in [0, T]$. `ClayStatements.LocalTetherClosingBound ν` asserts this for *all* smooth divergence-free finite-energy data. It implies `LocalFiniteTimeVorticityBound u₀ ν` (for each T_0 one ceiling on intervals $[0, T]$, $T \leq T_0$).

- `BKMGlobalContinuation u₀ ν` (hcont): if a Kato witness exists and `LocalFiniteTimeVorticityBound u₀ ν` holds, then there are u, p with $u(0) = u_0$, `NS_PDE u p ν`, $u(t)$ smooth for $t \geq 0$ and `IsPhysicalNSSolution u`. This packages Kato local existence with the Beale–Kato–Majda continuation criterion [3]; it is a classical-theorem input. *Caveat:* the classical statements are for H^s data, while the library's data class is $C^\infty \cap L^2$ on $\mathbb{R}^3$.

- `SmoothPhysicalUniqueness u₀ ν` (huniq): two local physical solutions from u_0 agree on the common interval; a classical-theorem input, used only for uniqueness clauses.

The global-path predicates `FrohmanianTetherRate`, `FrohmanianTetherBound` and `FrohmanianTetherClosingBound` are kept for auxiliary lemmas; the auxiliary predicates `UniformVorticityCeiling`, `FiniteTimeVorticityBound`, `BKMIntegrability`, `BKMContinuationCriterionFor`, `SmoothBelowTstarFor` and `ClosureBlueprint.RiccatiMajorantExists` now quantify only over physical solutions.

Theorem 8.4 (`TetheredLyapunov.conditional_regularity`; also `ClosureBlueprint.conditional_regularity_of_open_hyps`)**.** *Let w be a `KatoLocalWitness u₀ ν` and assume `BKMGlobalContinuation u₀ ν` and `LocalTetherBound u₀ ν`. Then there exist u, p with `NS_PDE u p ν`, $u(0) = u_0$, `IsPhysicalNSSolution u`, and $u(t)$ smooth and divergence-free for every $t \geq 0$ (and $M(t) \geq 0$).*

Theorem 8.5 (`ConditionalRegularity.conditional_regularity_of_open_hyps`)**.** *Under the hypotheses of Theorem 8.4 for $u_0(0)$ (stated for a time-dependent field $u_0(\cdot)$ that is divergence-free at all times, with $u_0(0)$ smooth and of finite energy), together with `SmoothPhysicalUniqueness (u0 0) ν`, there is a u with $u(0) = u_0(0)$, $u(t)$ smooth and satisfying `satisfies_NavierStokes` (physical solution of the equations) for $t \geq 0$, and unique among smooth fields with these properties.*

Theorem 8.6 (`frohmanian_tether_theorem`)**.** *Assume, for all u_0 and ν: (hKato) a Kato local witness for smooth divergence-free finite-energy u_0 and $\nu > 0$; (hcont) `BKMGlobalContinuation u₀ ν`; and (hDI) `LocalTetherRate u₀ ν`. Then there is a kernel $\mathfrak{T}_F$ such that: (1) every kernel with density equal to the canonical tether density equals $\mathfrak{T}_F$; and (2) for every $\nu > 0$ and every smooth divergence-free finite-energy u_0 there are u, p with `NS_PDE u p ν`, $u(0) = u_0$, and $u(t)$ smooth and divergence-free for all $t \geq 0$.*

Remark 8.7. Clause (2) is a global-existence statement *conditional on* (hDI) for all data, which is as strong as the open problem, together with the classical inputs (hKato) and (hcont); it is not a regularity theorem. The proof goes through the finite-interval comparison lemma (a time-uniform ceiling on all local solutions), then the continuation hypothesis. Version 0.1.1 had a clause assuming $\delta H/\delta\omega = u$; draft versions also had an antisymmetric clause, the hypotheses `hTlb`, `hInt`, `huniq_shift`, and then `hcrit`, `huniq` with a globally defined Kato witness. All were removed (Section 3.3).

Theorem 8.8 (Non-vacuity at zero data; `conditional_regularity_nonvacuous_at_zero`)**.** *For every $\nu > 0$ there is u with $u(0) = 0$, smooth and a physical solution for $t \geq 0$, unique among smooth physical solutions with zero data. It is proved from `kato_witness_zero` and the zero-data instances `smoothPhysicalUniqueness_zero`, `localTetherBound_zero`, `bkmGlobalContinuation_zero`. Also `stepField_not_physical`: a time-discontinuous step field, which would satisfy `NS_PDE` because `deriv` is 0 at non-differentiable points, is excluded by `IsPhysicalNSSolution`.*

This shows only that the conclusion form of Theorem 8.5 is consistent in the trivial case; it gives no information on nonzero data. Section 10 shows why the explicit periodic flows cannot replace the zero instance: for nonzero periodic data no `KatoLocalWitness` exists.

9 The periodic Clay statement and a conditional reduction

`ClayStatements.Clay_B_exact` formalizes a version of Fefferman's statement (B) [4]: for every $\nu > 0$ and every smooth, divergence-free, $\mathbb{Z}^3$-periodic u_0 on $\mathbb{R}^3$ there is a solution (u, p) that is C^∞ on $[0, \infty) \times \mathbb{R}^3$, solves the equations with zero forcing, has $u(0) = u_0$, and is periodic for all $t \geq 0$. (Fefferman's formulation also contains a bounded-energy condition; the formal statement is as quoted in Appendix A.7.)

Theorem 9.1 (clay_B_exact_of_periodic_tether_bound)**.** *If* `PeriodicTetherClosingBound ν` *and* `PeriodicBKMContinuation ν` *hold for every* $\nu > 0$*, then* `Clay_B_exact`*.*

`PeriodicTetherClosingBound` is a uniform vorticity bound for all periodic smooth data and `PeriodicBKMContinuation` says that such a bound yields a global periodic smooth solution. Together they are at least as strong as the conclusion; the reduction is a formal bookkeeping step and makes no progress on (B).

10 Explicit exact solutions: decaying shear and ABC flows

Coordinates are $x = (x_0, x_1, x_2)$ (Lean indices `0`, `1`, `2`). The module `ExplicitFlows` checks two classical moving solutions against the library's own equations `navier_stokes_eq` $\partial_t u + (u \cdot \nabla)u + \nabla p = \nu \Delta u$, $\operatorname{div} u = 0$ (for all $t \geq 0$), hence `NS_PDE` and `NS_PDE_On` for every T:

- *decaying shear* $u(t,x) = e^{-\nu k^2 t}\,(a \sin(kx_2), 0, 0)$ with constant pressure $p = 0$ (`shearFlow`, `shearFlow_apply`, `shearFlow_navier_stokes`, `shearFlow_NS_PDE_On`);
- *decaying ABC (Arnold–Beltrami–Childress) flow* $u(t,x) = e^{-\nu k^2 t}\,(A \sin kx_2 + C \cos kx_1,\ B \sin kx_0 + A \cos kx_2,\ C \sin kx_1 + B \cos kx_0)$ with pressure $p = -|u|^2/2$ (`abcFlow`, `abcProfile_apply`, `abcPressure`, `abcPressure₀_eq_norm`, `abcFlow_navier_stokes`, `abcFlow_NS_PDE_On`). Its profile is a curl eigenfield, $\nabla \times u = k\,u$ (`curl_abcProfile`).

Both statements hold for all real ν, k, a, A, B, C. Both come from one mechanism (`flow_navier_stokes`): a trigonometric profile V (`trigField`) has $\Delta V = -k^2 V$ (`laplacian_trigField`); if $\operatorname{div} V = 0$ and $(V \cdot \nabla)V + \nabla p_0 = 0$, then $u = \varphi(t)V$, $p = \varphi(t)^2 p_0$ with $\varphi' = -\nu k^2 \varphi$ solves the equations. For the shear profile $(V \cdot \nabla)V = 0$; for the ABC profile $(V \cdot \nabla)V = \nabla(|V|^2/2)$ and the sign of $p_0 = -|V|^2/2$ matches the library's convention $+\nabla p$ on the left (`abc_balance`). The time factor `decay` is $e^{-\nu k^2 t}$ for $t \geq 0$ and is continued affinely for $t < 0$, because `IsPhysicalNSSolutionOn` requires every solution to be affine for $t \leq 0$; the two-sided time derivative at $t = 0$ is then $-\nu k^2 u(0)$ (`hasDerivAt_decay`).

Genuine and nonzero. For $t \geq 0$, $a \neq 0$ and $k \neq 0$ the shear flow is not identically zero (`shearFlow_ne_zero`); the ABC flow at the origin equals $e^{-\nu k^2 t}(C, A, B)$ (`abcFlow_apply_zero`) and is nonzero unless $A = B = C = 0$ (`abcFlow_ne_zero`). Each time slice is C^∞ (`flow_smooth`), and the time-differentiability, continuity and affine-extension clauses of `IsPhysicalNSSolutionOn` hold (`flow_physical_clauses`).

Periodic class: genuine instances. With $k = 2\pi m$, $m \in \mathbb{Z}$, both flows are $\mathbb{Z}^3$-periodic and jointly C^∞ on $[0,\infty) \times \mathbb{R}^3$, with one-sided time derivative at $t = 0$, i.e. global smooth periodic solutions in the class `IsPeriodicSmoothSolution` of `Clay_B_exact` with zero forcing (`shearFlow_isPeriodicSmoothSolution`, `abcFlow_isPeriodicSmoothSolution`). So their initial data satisfy all hypotheses of `Clay_B_exact` and its conclusion holds *for these data* (`clay_B_instance_shear`, `clay_B_instance_abc`); the conclusion of `PeriodicBKMContinuation` therefore holds at these data without that hypothesis, and the binders of `PeriodicTetherClosingBound` and `PeriodicBKMContinuation` are non-vacuous there (`shearFlow_isPeriodicSmoothSolutionOn`, `abcFlow_isPeriodicSmoothSolutionOn`). This is a statement about two explicit families of data and is not a step toward `Clay_B_exact`; no Clay Millennium claim is made. The periodic closing bound `PeriodicTetherClosingBound` is *not* discharged at these data: it quantifies over every local smooth periodic solution from u_0, and uniqueness of such solutions is not formalized.

They are not Kato witnesses; no witness exists for periodic data. A `KatoLocalWitness` carries `IsPhysicalNSSolutionOn`, which requires $\int_{\mathbb{R}^3} \|u(t)\|^2 < \infty$. The library proves that no continuous, $\mathbb{Z}^3$-periodic, not identically zero field has $\|u\|^2$ integrable on $\mathbb{R}^3$ (`not_integrable_sq_of_periodic`: the set $\{\|u\|^2 \geq c\}$ contains infinitely many disjoint translates of a ball), hence KatoLocalWitness u_0 ν is empty for every such u_0 (`isEmpty_katoLocalWitness_of_periodic`), in particular for the shear and ABC data (`isEmpty_katoLocalWitness_shear`, `isEmpty_katoLocalWitness_abc`). The planned witness `kato_witness_shear` therefore cannot exist, and the $\mathbb{R}^3$ chain of Section 8 cannot be instantiated on these flows. Moreover, at such data the $\mathbb{R}^3$-class hypotheses `LocalTetherBound`, `SmoothPhysicalUniqueness` and `BKMGlobalContinuation` hold only *vacuously* (`localTetherBound_vacuous_of_periodic`, `smoothPhysicalUniqueness_vacuous_of_periodic`, `bkmGlobalContinuation_vacuous_of_periodic`): there is no local physical solution to quantify over, and the premise Nonempty(KatoLocalWitness u_0 ν) is false. These are recorded as an audit of the encoding and are *not* counted as discharged hypotheses. A nonzero finite-energy exact solution on $\mathbb{R}^3$ (and hence a nonzero `KatoLocalWitness`) is not constructed. (The library already contained the constant flows $u \equiv c$, `nonzero_smooth_ns_solution_exists`, which are stationary and also have infinite energy.)

11 What the Lean development does and does not prove

Proved (no further hypotheses). Symmetry of T and (C3); antisymmetry of $\{\cdot,\cdot\}_{\mathrm{cl}}$ and $\{H,H\}_{\mathrm{cl}} = 0$; Theorem 4.1 ($B(H,M) = 0$ for every symmetric B satisfying (C2)); the refutation Theorem 3.6 of $\delta H/\delta\omega = u$ and the non-uniqueness Theorem 3.7; `SDiffActionHyp` for the concrete permutation group; the refutations of Section 3.3; uniqueness of functional gradients; nondegeneracy, polarization on differentiable functionals, saturation, projection-dependence and the canonical density for T; the corrected weight condition `AEEnstrophyWeight` `TetherKernel` (Theorem 6.1); the transported (C1) for T under translations (Theorem 3.5); the finite-interval comparison lemma; the sanity lemma for `IsPhysicalNSSolution`; the comparison Lemma 8.1; the scaling identity and the scale non-invariance lemma; the zero-data facts and `stepField_not_physical`; the Fourier-symbol identities `PlanSteps.bsSymbol_div`, `PlanSteps.bsSymbol_curl` (mode-wise algebra, not connected to the real-space Biot–Savart integral); the explicit shear and ABC solutions, their non-triviality and periodic smoothness, and the non-existence of Kato witnesses for nonzero periodic data (Section 10).

Proved as implications from named, open hypotheses. Canonicity (from saturation and polarization); Casimir conservation (from `IsCoadjointMotion`); the conditional regularity theorems 8.4, 8.5, 8.6 (from rate or bound hypotheses on local solutions, Kato existence, Kato–BKM continuation and uniqueness); the Clay (B) reduction. The hypotheses are listed by category in Section 11.1.

Not proved, and not claimed.

- Global existence, smoothness or uniqueness for 3D Navier–Stokes with large data; any part of the Clay Millennium Problem.
- The cubic rate `LocalTetherRate` (or `FrohmanianTetherRate`), any closing bound, or any damping mechanism from the tether (under (C2) its damping coefficient would be 0).
- (C2) for T. The identification $\delta H/\delta\omega = u$ is refuted (Theorem 3.6); the classical identification $\delta H/\delta\omega = \psi$ (stream function) is not formalized.

- The transported form of (C1) for T under rotations or general volume-preserving diffeomorphisms (only translations are proved, Theorem 3.5; the fixed-functional form is refuted, Theorem 3.3). That the Euler flow is a coadjoint motion.
- A Jacobi identity for $\{\cdot,\cdot\}_{\mathfrak{T}}$ (the former conditional lemma was circular and is deleted), or a Poisson structure; the Jacobi identity for $\{\cdot,\cdot\}_{\mathrm{cl}}$ (cited only).
- Kato local existence [6], Beale–Kato–Majda [3], gluing of local solutions, parabolic smoothing, energy uniqueness: all are inputs, none is formalized.
- Anything on the quotient torus: `T3` is $\mathbb{R}^3$, and no diffeomorphism group of $\mathbb{T}^3$ is constructed.
- Derivation of T from (C1)–(C3): canonicity assumes diagonal agreement with T.
- Any nonzero Navier–Stokes solution of finite energy on $\mathbb{R}^3$, and any nonzero `KatoLocalWitness`; the sanity path for `IsPhysicalNSSolution` is not a solution. (Nonzero solutions of infinite energy are proved: the constant flows and the shear and ABC flows of Section 10.)
- Uniqueness of smooth periodic solutions, hence `PeriodicTetherClosingBound` even at the explicit data.

The physical motivations of earlier versions (fluid–gravity duality, Onsager's conjecture [5], metriplectic viscous completion) are not formalized and play no role in any statement here.

11.1 Remaining assumptions, by category

Every remaining assumption is an explicit Lean argument or a named `Prop`. None is an axiom.

(b) Classical results not in Mathlib.
(hKato) a `KatoLocalWitness` for smooth divergence-free finite-energy data (Kato local existence [6], equations on $[0,T]$ only); `BKMGlobalContinuation` (Kato local existence with Beale–Kato–Majda continuation [3]; the classical statements are for H^s data); `SmoothPhysicalUniqueness` (uniqueness of smooth solutions; uniqueness clauses only); `PeriodicBKMContinuation` (Clay reduction); `KatoUniquenessOnExistenceTimes` (auxiliary lemma `smoothBelowTstar_of_katoUniqueness` only); `IsCoadjointMotion` for the Euler flow (Arnold [1]).

(d) Equivalent in strength to the open problem.
`LocalTetherRate` (the cubic rate on every local physical solution; it also asks for time-differentiability of $\|\omega(t)\|_{L^\infty}$, which can fail for genuine flows); `LocalTetherBound` and `ClayStatements.LocalTetherClosingBound` (uniform vorticity bounds on local solutions); `ClayStatements.PeriodicTetherClosingBound` and `PeriodicDirectionDepletionBound`; the global-path forms `FrohmanianTetherRate`, `FrohmanianTetherBound`, `FrohmanianTetherClosingBound` and `RiccatiMajorantExists` (auxiliary lemmas only). The tether does not supply any of these.

(c) Kernel-class conditions on a competitor B.
In the canonicity theorems: symmetry, `PolarizesOnDifferentiable B`, `SaturatesTetherQuadratic B`, `h_proj`, `h_density`, `h_weight : AEEnstrophyWeight B` (both for locally integrable densities; and (C1)–(C3) for B, unused). They define the class compared with T.

Open for T itself.
(C2) at regular ω (`DegenerateWRTKineticEnergy TetherKernel`), assumed in Corollary 4.2 and expected to fail; transport-invariance under rotations and general diffeomorphisms (not

formalized). Settled since the previous draft: `AEEnstrophyWeight TetherKernel` is proved (Theorem 6.1); fixed-functional (C1) is refuted (Theorem 3.3); transported (C1) for translations is proved (Theorem 3.5).

Vacuity at periodic data. Because `IsPhysicalNSSolutionOn` requires finite energy on $\mathbb{R}^3$, `LocalTetherBound`, `SmoothPhysicalUniqueness` and `BKMGlobalContinuation` hold trivially at every continuous nonzero $\mathbb{Z}^3$-periodic u_0 and `KatoLocalWitness` is empty there (Section 10). They are not discharged by the explicit flows; the list above is unchanged by version 0.1.4.

Hypotheses of auxiliary lemmas. Re-derived from the source (every theorem binder whose type is a named `Prop` of the library): besides the items above, auxiliary lemmas that are not stated in this paper take the following named, unproved `Prop`s as explicit hypotheses: `BKMContinuationCriterion`, `BKMContinuationCriterionFor`, `LocalExistenceTime`, `SmoothBelowTstar`, `SmoothBelowTstarFor`, `PathBKMIntegrable`, `TstarIsTop`, `KatoUniformRestart`, `UniformVorticityCeiling`, `FiniteTimeVorticityBound`, `PhysicalUniformVorticityCeiling`, `PhysicalFiniteTimeVorticityBound`, `LocalFiniteTimeVorticityBound`, `ClassicalStretchingSource`, `CorrectedMaxRate`, `SymplecticQuarticStretchingSinkAtMax`, `Clay_B_exact_with_energy`, `ZeroDerivAlongEnergy`, and the regularity side conditions `VorticityTransportRegularity`, `SpatialMaxVorticityEnvelope`, `PeriodicCellPhysicalNSSolution`, `VolumePreservingCoadjointAction`. None of them is used by a theorem stated in Sections 3–10; each is an explicit argument, never an axiom.

11.2 Mathlib declarations used; project-local definitions

Mathlib declarations used directly by the new modules. `ContDiff`, `ContDiffBump` (`ContDiffBump.one_of_mem_closedBall`, `ContDiffBump.tsupport_eq`), `fderiv`, `ContDiffAt.isSymmSndFDerivAt`, `tsupport`, `HasCompactSupport`, `LocallyIntegrable` (`LocallyIntegrable.integrable_smul_right_of_hasCompactSupport`), `IsOpen.ae_eq_zero_of_integral_contDiff_smul_eq_zero`, `TopologicalSpace.isOpen_iUnion_countable`, `EuclideanSpace.single`, `EuclideanSpace.proj`, the `PiLp`/`WithLp` API, `Lp.compMeasurePreserving`$_{\mathrm{ii}}$, `Lp.coeFn_compMeasurePreserving`, `measurePreserving_sub_right`, `integral_add_right_eq_self`, `integral_sub_right_eq_self`, `map_mem_closure`, `Submodule.topologicalClosure`, `LinearIsometry.inner_map_map`, `Equiv.addRight`, `Subgroup`, `crossProduct`, `QuotientAddGroup` (Vitali construction). Module `ExplicitFlows`: `Real.sin`, `Real.cos`, `Real.exp`, `HasFDerivAt`, `HasDerivAt`, `HasDerivWithinAt.union`, `contDiff_piLp`, `Real.sin_add_int_mul_two_pi`, `Real.cos_add_int_mul_two_pi`, `EuclideanSpace.norm_sq_eq`, `PiLp.norm_single`, `Integrable.measure_norm_ge_lt_top`, `Measure.addHaar_ball_center`, `Metric.measure_ball_pos`, `Metric.ball_disjoint_ball`, `measure_iUnion`, `ENNReal.tsum_const_eq_top_of_ne_zero`.

Project-local names proved equal to Mathlib declarations (module `MathlibAgreement`, all by `rfl`): `NavierStokes3D.volume` = `MeasureTheory.volume`; `pairing` = the real inner product; `eb i` = `EuclideanSpace.single i 1`; `pd` = `fderiv` along a basis vector; `NavierStokes3D.div` = the sum of `fderiv` partials; `cross` = `crossProduct`; `translate` = `Equiv.addRight`; `l2Translate` = `Lp.compMeasurePreserving`$_{\mathrm{ii}}$; `Enstrophy` = $\int \|\omega\|^2$.

Genuinely project-local definitions (no Mathlib counterpart): `curlVal` (curl of He_2), `CoadjointOrbit` (pointwise divergence-free fields), `IsTestDirection`, `testFields`, `DivFreeL2`, `IsFunctionalGradient`, `FunctionalDerivative`, `BiotSavart` and `biotSavartKernel`, `Pi_u`,

`projCoef`, `TetherKernel`, the concrete group `ConcreteSDiff.G`, and the predicates (C1)–(C3), `InvariantUnderTransportedCoadjointAction`; in `ExplicitFlows`: `tp`, `tq` (trigonometric pairs), `trigField`, `decay` (exponential continued affinely to $t < 0$), `flow`, `flowPressure`, `shearProfile`, `shearFlow`, `abcProfile`, `abcFlow`, `abcPressure`.

Remaining duplicates, not refactored in this version. `NavierStokes3D.div` could be stated via the trace of `fderiv`; `Pi_u` is a pointwise formula rather than Mathlib's `Submodule.orthogonalProjection` on L^2; the $\mathbb{Z}^3$-periodicity predicate `IsZ3Periodic` is not stated through Mathlib's `Function.Periodic` along each coordinate direction; the Kronecker delta `kd` in `ExplicitFlows` is the coordinate `EuclideanSpace.single i 1 j` (`single_apply_kd`). Existing working code was left unchanged.

A Exact Lean statements

Quoted verbatim from the repository by `scripts/extract_statements.py`; proof bodies are replaced by `...`. Each block starts with its file and line.

A.1 Setting

```
-- FrohmanianTether/ArnoldGeometric.lean, line 167
public structure CoadjointOrbit where
  val : T3 → EuclideanSpace ℝ (Fin 3)
  property : ∀ x, NavierStokes3D.div val x = 0
```

```
-- FrohmanianTether/ArnoldGeometric.lean, line 309
@[expose] public def IsTestDirection (ε : CoadjointOrbit) : Prop :=
  ContDiff ℝ (⊤ : ℕ∞) ε.val ∧ HasCompactSupport ε.val
```

```
-- FrohmanianTether/ArnoldGeometric.lean, line 328
@[expose] public noncomputable def DivFreeL2 : Submodule ℝ L2Field := (Submodule.span ℝ
↪  testFields).topologicalClosure
```

```
-- FrohmanianTether/ArnoldGeometric.lean, line 337
@[expose] public def IsFunctionalGradient (F : CoadjointOrbit → ℝ) (ω : CoadjointOrbit) (g :
↪  L2Field) : Prop :=
  g ∈ DivFreeL2 ∧ ∃ hω : IsRegularVorticity ω, ∀ ε (hε : IsTestDirection ε),
    HasDerivAt (fun t => F (orbitLine ω ε hω hε.regular t)) (inner ℝ g (testToL2 ε hε)) 0
```

```
-- FrohmanianTether/ArnoldGeometric.lean, line 353
public theorem isFunctionalGradient_unique {F : CoadjointOrbit → ℝ} {ω : CoadjointOrbit} {g₁
↪  g₂ : L2Field}
    (h₁ : IsFunctionalGradient F ω g₁) (h₂ : IsFunctionalGradient F ω g₂) : g₁ = g₂ := ...
```

```
-- FrohmanianTether/ArnoldGeometric.lean, line 367
@[expose] public noncomputable def FunctionalDerivative (F : CoadjointOrbit → ℝ) (ω :
↪  CoadjointOrbit) : T3 → EuclideanSpace ℝ (Fin 3) :=
  if h : ∃ g, IsFunctionalGradient F ω g then ⇑(Classical.choose h) else 0
```

```
-- FrohmanianTether/ArnoldGeometric.lean, line 1014
public noncomputable def KineticEnergyHamiltonian (ω : CoadjointOrbit) : ℝ :=
  (1/2) * ∫ x, ‖BiotSavart ω.val x‖^2 ∂(volume)
```

```
-- FrohmanianTether/NS_Equations.lean, line 2149
@[expose] public def navier_stokes_eq (u : TimeDependentVelocity) (p : TimeDependentPressure)
    (ν : ℝ) : Prop :=
  ∀ t ≥ (0 : ℝ), ∀ x : T3,
    time_deriv u t x + convective (u t) (u t) x + pressureGradient (p t) x =
        ν • laplacian (u t) x ∧
      div (u t) x = 0
```

```
-- FrohmanianTether/NS_Equations.lean, line 2270
@[expose] public def IsPhysicalNSSolution (u : TimeDependentVelocity) : Prop :=
  (∀ t ≥ (0 : ℝ), ∀ x : T3, DifferentiableAt ℝ (fun s => u s x) t) ∧
  (∀ t ≥ (0 : ℝ), Continuous (u t)) ∧
  (∀ t ≥ (0 : ℝ), Integrable (fun x : T3 => ‖u t x‖ ^ 2)) ∧
  (∀ t ≥ (0 : ℝ), (∫ x, ‖u t x‖ ^ 2 ∂volume) ≤ ∫ x, ‖u 0 x‖ ^ 2 ∂volume) ∧
  (∃ v₀ : VelocityField, ∀ t ≤ (0 : ℝ), u t = u 0 + t • v₀)
```

```
-- FrohmanianTether/PhysicalSolutionSanity.lean, line 144
public theorem exists_physical_time_deriv_ne_zero :
    ∃ u : TimeDependentVelocity, IsPhysicalNSSolution u ∧
      ∃ x : T3, deriv (fun s => u s x) 0 ≠ 0 := ...
```

```
-- FrohmanianTether/PhysicalSolutionSanity.lean, line 157
public theorem not_forall_physical_time_deriv_zero :
    ¬ ∀ u : TimeDependentVelocity, IsPhysicalNSSolution u →
      ∀ x : T3, deriv (fun s => u s x) 0 = 0 := ...
```

```
-- FrohmanianTether/NS_Equations.lean, line 2293
@[expose] public def NS_PDE_On (u : TimeDependentVelocity) (p : TimeDependentPressure) (ν T :
↪  ℝ) :
    Prop :=
  ∀ t ∈ Set.Icc (0 : ℝ) T, ∀ x : T3,
    time_deriv u t x + convective (u t) (u t) x + pressureGradient (p t) x =
        ν • laplacian (u t) x ∧
      div (u t) x = 0
```

```
-- FrohmanianTether/NS_Equations.lean, line 2302
@[expose] public def IsPhysicalNSSolutionOn (u : TimeDependentVelocity) (T : ℝ) : Prop :=
  (∀ t ∈ Set.Icc (0 : ℝ) T, ∀ x : T3, DifferentiableAt ℝ (fun s => u s x) t) ∧
  (∀ t ∈ Set.Icc (0 : ℝ) T, Continuous (u t)) ∧
  (∀ t ∈ Set.Icc (0 : ℝ) T, Integrable (fun x : T3 => ‖u t x‖ ^ 2)) ∧
  (∀ t ∈ Set.Icc (0 : ℝ) T, (∫ x, ‖u t x‖ ^ 2 ∂volume) ≤ ∫ x, ‖u 0 x‖ ^ 2 ∂volume) ∧
  (∃ v₀ : VelocityField, ∀ t ≤ (0 : ℝ), u t = u 0 + t • v₀)
```

```
-- FrohmanianTether/NS_Equations.lean, line 2329
@[expose] public def IsLocalPhysicalSolution (ν T : ℝ) (u₀ : VelocityField)
    (u : TimeDependentVelocity) (p : TimeDependentPressure) : Prop :=
  u 0 = u₀ ∧ NS_PDE_On u p ν T ∧ (∀ t ∈ Set.Icc (0 : ℝ) T, ContDiff ℝ (⊤ : ℕ∞) (u t)) ∧
    IsPhysicalNSSolutionOn u T
```

```
-- FrohmanianTether/NS_Equations.lean, line 2486
public structure KatoLocalWitness (u₀ : VelocityField) (ν : ℝ) where
  T : ℝ
  Tpos : 0 < T
  u : TimeDependentVelocity
```

```
  p : TimeDependentPressure
  smooth : ∀ t ∈ Set.Icc (0 : ℝ) T, ContDiff ℝ (⊤ : ℕ∞) (u t)
  init : u 0 = u₀
  pde : NS_PDE_On u p ν T
  phys : IsPhysicalNSSolutionOn u T
```

A.2 Tether, (C1)–(C3), obstruction, canonicity

```
-- FrohmanianTether/ForMathlib/Projection.lean, line 35
@[expose] public noncomputable def Pi_u (u v : VelocityField) : VelocityField :=
  fun x => v x - (
    (∫ y, pairing (v y) (u y) ∂volume) /
    (∫ y, ‖u y‖ ^ 2 ∂volume)
  ) • u x
```

```
-- FrohmanianTether/SymplecticTether.lean, line 198
@[expose] public noncomputable def TetherKernel (ω : CoadjointOrbit) (F G : Functional) : ℝ :=
  -kappa * Enstrophy ω * ∫ x,
      inner ℝ
        (Pi_u (velocity_from_vorticity ω) (FunctionalDerivative F ω) x)
        (Pi_u (velocity_from_vorticity ω) (FunctionalDerivative G ω) x)
    ∂volume
```

```
-- FrohmanianTether/SymplecticTether.lean, line 205
public noncomputable def TetheredBracket (F G : Functional) (ω : CoadjointOrbit) : ℝ :=
  -- The Frohmanian Symplectic Tether bracket: the classical Arnold Lie–Poisson bracket
  -- plus the minimal quadratic correction (the tether kernel) that satisfies (C1)–(C3).
  -- Canonicity is proved only inside the restricted kernel class and under the explicit
  -- hypotheses of `uniqueness_of_minimal_tether` (Uniqueness.lean); it is NOT an
  -- unconditional uniqueness statement. The correction is symmetric, so this bracket is
  -- not antisymmetric.
  ClassicalBracket F G ω + TetherKernel ω F G
```

```
-- FrohmanianTether/SymplecticTether.lean, line 222
@[expose] public def InvariantUnderCoadjointAction (SDiff : Type*) [Group SDiff]
    [MulAction SDiff CoadjointOrbit] (B : CoadjointOrbit → Functional → Functional → ℝ) : Prop
    ↪  :=
  -- (C1): The correction B is invariant under the coadjoint action of SDiff(T³).
  -- For all g, the value of the correction is unchanged when the vorticity is transformed by
  -- the coadjoint action; the test functionals F, G are NOT transformed.
  ∀ (g : SDiff) (F G : Functional) (ω : CoadjointOrbit),
    B (CoadjointAction g ω) F G = B ω F G
```

```
-- FrohmanianTether/SymplecticTether.lean, line 230
@[expose] public def DegenerateWRTKineticEnergy (B : CoadjointOrbit → Functional → Functional
↪  → ℝ) : Prop :=
  -- (C2): The correction is degenerate with respect to the kinetic-energy Hamiltonian H.
  -- The Hamiltonian vector field generated by H is exactly the classical one (no modification
  -- to the reversible Euler dynamics). This is the energy-preserving degeneracy on energy.
  ∀ (F : Functional) (ω : CoadjointOrbit),
    B ω F KineticEnergyHamiltonian = 0
```

```
-- FrohmanianTether/SymplecticTether.lean, line 238
@[expose] public def ProducesControllableNegativeFeedback
    (B : CoadjointOrbit → Functional → Functional → ℝ) : Prop :=
  ∀ (F : Functional) (ω : CoadjointOrbit),
```

```
    B ω F F ≤
      -kappa * Enstrophy ω * ∫ x,
          ‖Pi_u (velocity_from_vorticity ω) (FunctionalDerivative F ω) x‖ ^ 2
        ∂volume
```

```
-- FrohmanianTether/SymplecticTether.lean, line 261
public theorem tetherKernel_comm (ω : CoadjointOrbit) (F G : Functional) :
    TetherKernel ω F G = TetherKernel ω G F := ...
```

```
-- FrohmanianTether/SymplecticTether.lean, line 292
public theorem tetherKernel_C3 (F : Functional) (ω : CoadjointOrbit) :
    TetherKernel ω F F ≤
      -kappa * Enstrophy ω * ∫ x,
          ‖Pi_u (velocity_from_vorticity ω) (FunctionalDerivative F ω) x‖ ^ 2
        ∂volume := ...
```

```
-- FrohmanianTether/SymplecticTether.lean, line 623
public theorem tetherKernel_C2_of_no_gateaux
    (ω : CoadjointOrbit)
    (h : ¬ ∃ dH, IsFunctionalGradient KineticEnergyHamiltonian ω dH)
    (F : Functional) :
    TetherKernel ω F KineticEnergyHamiltonian = 0 := ...
```

```
-- FrohmanianTether/KineticEnergyDerivative.lean, line 286
theorem not_forall_functionalDerivative_kineticEnergy_eq_velocity :
    ¬ ∀ ω : CoadjointOrbit,
      FunctionalDerivative KineticEnergyHamiltonian ω = velocity_from_vorticity ω := ...
```

```
-- FrohmanianTether/KineticEnergyDerivative.lean, line 303
theorem not_unique_isGateauxRepresentative (F : CoadjointOrbit → ℝ) (ω : CoadjointOrbit)
    (d : T3 → EuclideanSpace ℝ (Fin 3)) (hd : IsGateauxRepresentative F ω d) :
    ∃ d', IsGateauxRepresentative F ω d' ∧ d' ≠ d := ...
```

```
-- FrohmanianTether/KineticEnergyDerivative.lean, line 328
theorem not_unique_velocity_representative (ω : CoadjointOrbit) :
    ¬ (IsGateauxRepresentative KineticEnergyHamiltonian ω (velocity_from_vorticity ω) ∧
      ∀ dH, IsGateauxRepresentative KineticEnergyHamiltonian ω dH →
        dH = velocity_from_vorticity ω) := ...
```

```
-- FrohmanianTether/ArnoldGeometric.lean, line 1102
public theorem isVelocityGateauxRepresentative_kineticEnergy_at
    (ω ε : CoadjointOrbit)
    (hω : ∀ i x, DifferentiableAt ℝ (fun y => ω.val y i) x)
    (hε : ∀ i x, DifferentiableAt ℝ (fun y => ε.val y i) x)
    (hkerω : ∀ x, Integrable
      (fun y => biotSavartKernel x y • cross (ω.val y) (x - y)))
    (hkerε : ∀ x, Integrable
      (fun y => biotSavartKernel x y • cross (ε.val y) (x - y)))
    (hF_meas : ∀ᶠ t in nhds (0 : ℝ),
      AEStronglyMeasurable
        (fun x => (1 / 2 : ℝ) *
          ‖BiotSavart ω.val x + t • BiotSavart ε.val x‖ ^ 2)
        NavierStokes3D.volume)
    (hF_int : Integrable
      (fun x => (1 / 2 : ℝ) * ‖BiotSavart ω.val x‖ ^ 2)
```

```
      NavierStokes3D.volume)
    (hbound : Integrable
      (fun x => ‖BiotSavart ω.val x‖ * ‖BiotSavart ε.val x‖ +
        ‖BiotSavart ε.val x‖ ^ 2)
      NavierStokes3D.volume)
    (hinner : AEStronglyMeasurable
      (fun x => inner ℝ (BiotSavart ω.val x) (BiotSavart ε.val x))
      NavierStokes3D.volume) :
    HasDerivAt
      (fun t : ℝ =>
        KineticEnergyHamiltonian
          (fun x => ω.val x + t • ε.val x,
            fun x => orbit_add_smul_div_free ω ε t hω hε x))
      (∫ x, pairing (velocity_from_vorticity ω x) (BiotSavart ε.val x)
        ∂volume) 0 := ...
```

```
-- FrohmanianTether/ArnoldGeometric.lean, line 219
public structure SDiffActionHyp (SDiff : Type*) [Group SDiff] [MulAction SDiff CoadjointOrbit]
↪  :
    Prop where
  casimir_invariant : ∀ (f : ℝ → ℝ) (g : SDiff) (ω : CoadjointOrbit),
    (∫ x, f ‖(g • ω).val x‖ ∂volume) = ∫ x, f ‖ω.val x‖ ∂volume
```

```
-- FrohmanianTether/ConcreteSDiff.lean, line 98
theorem sdiffActionHyp : SDiffActionHyp G := ...
```

```
-- FrohmanianTether/ConcreteSDiff.lean, line 87
theorem casimir_invariant (f : ℝ → ℝ) (g : G) (ω : CoadjointOrbit) :
    (∫ x, f ‖(g • ω).val x‖ ∂volume) = ∫ x, f ‖ω.val x‖ ∂volume := ...
```

```
-- FrohmanianTether/Combined.lean, line 44
theorem left_H_zero_of_C2_symm
    (B : CoadjointOrbit → Functional → Functional → ℝ)
    (h_sym : ∀ ω F G, B ω F G = B ω G F) (hC2 : DegenerateWRTKineticEnergy B)
    (ω : CoadjointOrbit) (M : Functional) :
    B ω KineticEnergyHamiltonian M = 0 := ...
```

```
-- FrohmanianTether/Combined.lean, line 52
theorem tetherKernel_left_H_zero (hC2 : DegenerateWRTKineticEnergy TetherKernel)
    (ω : CoadjointOrbit) (M : Functional) :
    TetherKernel ω KineticEnergyHamiltonian M = 0 ∧ TetherKernel ω M KineticEnergyHamiltonian
    ↪  = 0 := ...
```

```
-- FrohmanianTether/ArnoldGeometric.lean, line 1211
public theorem classicalBracket_antisymm (F G : CoadjointOrbit → ℝ) (ω : CoadjointOrbit) :
    ClassicalBracket F G ω = - ClassicalBracket G F ω := ...
```

```
-- FrohmanianTether/Uniqueness.lean, line 226
public def SaturatesTetherQuadratic
    (B : CoadjointOrbit → Functional → Functional → ℝ) : Prop :=
  ∀ (F : Functional) (ω : CoadjointOrbit), B ω F F = TetherKernel ω F F
```

```
-- FrohmanianTether/Uniqueness.lean, line 231
public def Polarizes
```

```
    (B : CoadjointOrbit → Functional → Functional → ℝ) : Prop :=
  ∀ (ω : CoadjointOrbit) (F G : Functional),
    2 * B ω F G =
      B ω (fun ω' => F ω' + G ω') (fun ω' => F ω' + G ω') -
        B ω F F - B ω G G
```

```
-- FrohmanianTether/Uniqueness.lean, line 241
@[expose] public def PolarizesOnDifferentiable
    (B : CoadjointOrbit → Functional → Functional → ℝ) : Prop :=
  ∀ (ω : CoadjointOrbit) (F G : Functional),
    (∃ g, IsFunctionalGradient F ω g) → (∃ g, IsFunctionalGradient G ω g) →
    2 * B ω F G =
      B ω (fun ω' => F ω' + G ω') (fun ω' => F ω' + G ω') - B ω F F - B ω G G
```

```
-- FrohmanianTether/Uniqueness.lean, line 250
public theorem tetherKernel_polarizesOnDifferentiable :
    PolarizesOnDifferentiable TetherKernel := ...
```

```
-- FrohmanianTether/TetherNondegeneracy.lean, line 320
public theorem tetherKernel_nondegenerate :
    ∃ (ω φ : CoadjointOrbit) (hφ : IsTestDirection φ), IsRegularVorticity ω ∧
      TetherKernel ω (linFun φ) (linFun φ) ≠ 0 := ...
```

```
-- FrohmanianTether/Uniqueness.lean, line 605
public theorem tetherKernel_kernel_conditions :
    SaturatesTetherQuadratic TetherKernel ∧
    (∀ ω (F F' G : Functional),
      Pi_u (velocity_from_vorticity ω) (FunctionalDerivative F ω) =
        Pi_u (velocity_from_vorticity ω) (FunctionalDerivative F' ω) →
        TetherKernel ω F G = TetherKernel ω F' G) ∧
    HasTetherKernelDensity TetherKernel canonicalTetherDensity ∧
    PolarizesOnDifferentiable TetherKernel ∧
    (∃ ω F, TetherKernel ω F F ≠ 0) := ...
```

```
-- FrohmanianTether/Uniqueness.lean, line 517
@[expose] public def AEEnstrophyWeightUnrestricted
    (B : CoadjointOrbit → Functional → Functional → ℝ) : Prop :=
  ∀ α, HasTetherKernelDensity B α → ∃ c : ℝ, ∀ ω, IsRegularVorticity ω →
    ∀ᵐ x ∂(MeasureTheory.volume : Measure T3), α ω x = -c * Enstrophy ω
```

```
-- FrohmanianTether/Uniqueness.lean, line 528
@[expose] public def AEEnstrophyWeight
    (B : CoadjointOrbit → Functional → Functional → ℝ) : Prop :=
  ∀ α, HasTetherKernelDensity B α →
    (∀ ω, IsRegularVorticity ω → LocallyIntegrable (α ω) (MeasureTheory.volume : Measure T3)) →
    ∃ c : ℝ, ∀ ω, IsRegularVorticity ω →
      ∀ᵐ x ∂(MeasureTheory.volume : Measure T3), α ω x = -c * Enstrophy ω
```

```
-- FrohmanianTether/TetherWeight.lean, line 246
public theorem tetherKernel_has_locallyIntegrable_density :
    ∃ α, HasTetherKernelDensity TetherKernel α ∧
      ∀ ω, IsRegularVorticity ω → LocallyIntegrable (α ω) (MeasureTheory.volume : Measure T3)
      ↪  := ...
```

```
-- FrohmanianTether/Uniqueness.lean, line 399
@[expose] public def FactorsThroughProjection
    (B : CoadjointOrbit → Functional → Functional → ℝ) : Prop :=
  ∀ ω : CoadjointOrbit, ∃ Q : (T3 → EuclideanSpace ℝ (Fin 3)) → (T3 → EuclideanSpace ℝ (Fin
  ↪  3)) → ℝ,
    (∀ a b, Q a b = Q b a) ∧
    ∀ F G : Functional, B ω F G =
      Q (Pi_u (velocity_from_vorticity ω) (FunctionalDerivative F ω))
        (Pi_u (velocity_from_vorticity ω) (FunctionalDerivative G ω))
```

```
-- FrohmanianTether/Uniqueness.lean, line 47
@[expose] public def HasTetherKernelDensity
    (B : CoadjointOrbit → Functional → Functional → ℝ)
    (α : CoadjointOrbit → T3 → ℝ) : Prop :=
  ∀ (ω : CoadjointOrbit) (F G : Functional),
    B ω F G =
      ∫ x,
        α ω x *
          inner ℝ
            (Pi_u (velocity_from_vorticity ω) (FunctionalDerivative F ω) x)
            (Pi_u (velocity_from_vorticity ω) (FunctionalDerivative G ω) x)
        ∂volume
```

```
-- FrohmanianTether/Uniqueness.lean, line 653
public theorem uniqueness_of_minimal_tether
    (SDiff : Type*) [Group SDiff] [MulAction SDiff CoadjointOrbit]
    (_hyp : SDiffActionHyp SDiff)
    (B : CoadjointOrbit → Functional → Functional → ℝ)
    (h_sym : ∀ ω F G, B ω F G = B ω G F)
    (hC1 : InvariantUnderCoadjointAction SDiff B)
    (hC2 : DegenerateWRTKineticEnergy B)
    (hC3 : ProducesControllableNegativeFeedback B)
    (h_sat : SaturatesTetherQuadratic B)
    (h_polarB : PolarizesOnDifferentiable B) :
    ∀ (ω : CoadjointOrbit) (F G : Functional), (∃ g, IsFunctionalGradient F ω g) →
      (∃ g, IsFunctionalGradient G ω g) → B ω F G = TetherKernel ω F G := ...
```

```
-- FrohmanianTether/Uniqueness.lean, line 633
public theorem uniqueness_of_minimal_tether_core
    (B : CoadjointOrbit → Functional → Functional → ℝ)
    (h_sym : ∀ ω F G, B ω F G = B ω G F)
    (h_proj : ∀ ω (F F' G : Functional),
      Pi_u (velocity_from_vorticity ω) (FunctionalDerivative F ω) =
        Pi_u (velocity_from_vorticity ω) (FunctionalDerivative F' ω) → B ω F G = B ω F' G)
    (h_density : FactorsThroughProjection B → ∃ α, HasTetherKernelDensity B α ∧
      ∀ ω, IsRegularVorticity ω → LocallyIntegrable (α ω) (MeasureTheory.volume : Measure T3))
    (h_weight : AEEnstrophyWeight B)
    (h_sat : SaturatesTetherQuadratic B) :
    ∀ ω F G, B ω F G = TetherKernel ω F G := ...
```

```
-- FrohmanianTether/PlanSteps.lean, line 61
def IsCoadjointMotion (SDiff : Type*) [Group SDiff] [MulAction SDiff CoadjointOrbit]
    (ω : ℝ → CoadjointOrbit) : Prop :=
  ∃ g : ℝ → SDiff, ∀ t, ω t = g t • ω 0
```

```
-- FrohmanianTether/PlanSteps.lean, line 67
theorem casimir_conserved {SDiff : Type*} [Group SDiff] [MulAction SDiff CoadjointOrbit]
```

```
    (H : SDiffActionHyp SDiff) (ω : ℝ → CoadjointOrbit) (hω : IsCoadjointMotion SDiff ω)
    (f : ℝ → ℝ) (t : ℝ) : Casimir f (ω t) = Casimir f (ω 0) := ...
```

A.3 Concentrated test fields and the weight condition

```
-- FrohmanianTether/TestFieldFamily.lean, line 38
@[expose] public def curlVal (H : T3 → ℝ) (y : T3) : EuclideanSpace ℝ (Fin 3) :=
  pd H 1 y • eb 0 - pd H 0 y • eb 1
```

```
-- FrohmanianTether/TestFieldFamily.lean, line 76
public theorem div_curlVal {H : T3 → ℝ} (hH : ContDiff ℝ ∞ H) (x : T3) :
    NavierStokes3D.div (curlVal H) x = 0 := ...
```

```
-- FrohmanianTether/TestFieldFamily.lean, line 108
public theorem curlOrbit_test {H : T3 → ℝ} (hH : ContDiff ℝ ∞ H) (hc : HasCompactSupport H) :
    IsTestDirection (curlOrbit H hH) := ...
```

```
-- FrohmanianTether/TestFieldFamily.lean, line 117
@[expose] public def concField (x : T3) {r : ℝ} (hr : 0 < r) : CoadjointOrbit :=
  curlOrbit (concBump x hr) (concBump x hr).contDiff
```

```
-- FrohmanianTether/TestFieldFamily.lean, line 120
public theorem concField_test (x : T3) {r : ℝ} (hr : 0 < r) : IsTestDirection (concField x hr)
↪  := ...
```

```
-- FrohmanianTether/TestFieldFamily.lean, line 123
public theorem tsupport_concField_subset (x : T3) {r : ℝ} (hr : 0 < r) :
    tsupport (concField x hr).val ⊆ Metric.ball x r := ...
```

```
-- FrohmanianTether/TestFieldFamily.lean, line 130
public theorem concField_ne_zero (x : T3) {r : ℝ} (hr : 0 < r) :
    ∃ y, (concField x hr).val y ≠ 0 := ...
```

```
-- FrohmanianTether/TetherWeightProof.lean, line 49
@[expose] public def projCoef (ω : CoadjointOrbit) (v : T3 → EuclideanSpace ℝ (Fin 3)) : ℝ :=
  (∫ y, pairing (v y) (velocity_from_vorticity ω y) ∂NavierStokes3D.volume) /
    (∫ y, ‖velocity_from_vorticity ω y‖ ^ 2 ∂NavierStokes3D.volume)
```

```
-- FrohmanianTether/TetherWeightProof.lean, line 89
public theorem integral_beta_eq_zero {α : CoadjointOrbit → T3 → ℝ}
    (hα : HasTetherKernelDensity TetherKernel α) {ω : CoadjointOrbit} (hω : IsRegularVorticity
    ↪  ω)
    (hloc : LocallyIntegrable (α ω) (MeasureTheory.volume : Measure T3))
    {a b : CoadjointOrbit} (ha : IsTestDirection a) (hb : IsTestDirection b)
    (ha0 : projCoef ω a.val = 0) (hb0 : projCoef ω b.val = 0) :
    ∫ x, (α ω x + kappa * Enstrophy ω) * inner ℝ (a.val x) (b.val x) ∂MeasureTheory.volume = 0
    ↪  := ...
```

```
-- FrohmanianTether/TetherWeightProof.lean, line 218
public theorem good_of_two_correctors {β : T3 → ℝ} {lam : (T3 → EuclideanSpace ℝ (Fin 3)) → ℝ}
    (HV : KillsOrth β lam) (Hlin : CoefAdditive lam) {x y w : T3} (hxy : x ≠ y) (hxw : x ≠ w)
    (hyw : y ≠ w) (hy : CoefPt lam y) (hw : CoefPt lam w) : GoodPt β x := ...
```

```
-- FrohmanianTether/TetherWeightProof.lean, line 259
public theorem bad_finite {β : T3 → ℝ} {lam : (T3 → EuclideanSpace ℝ (Fin 3)) → ℝ}
    (HV : KillsOrth β lam) (Hlin : CoefAdditive lam) : {x | ¬ GoodPt β x}.Finite := ...
```

```
-- FrohmanianTether/TetherWeightProof.lean, line 287
public theorem pointwise_identity {g χ : T3 → ℝ} (hg : ContDiff ℝ (⊤ : ℕ∞) g)
    (hχ : ∀ z ∈ tsupport g, χ =ᶠ[nhds z] fun _ => 1) (z : T3) :
    pd (fun y => y 0 * g y) 0 z * pd (fun y => y 0 * χ y) 0 z +
        pd (fun y => y 0 * g y) 1 z * pd (fun y => y 0 * χ y) 1 z -
      (pd g 0 z * pd (fun y => y 0 * y 0 / 2 * χ y) 0 z +
        pd g 1 z * pd (fun y => y 0 * y 0 / 2 * χ y) 1 z) = g z := ...
```

```
-- FrohmanianTether/TetherWeightProof.lean, line 323
public theorem ae_zero_on_good_ball {β : T3 → ℝ}
    (hβ : LocallyIntegrable β (MeasureTheory.volume : Measure T3)) {x : T3} {ε : ℝ} (hε : 0 <
    ↪  ε)
    (hgood : ∀ a b : CoadjointOrbit, IsTestDirection a → IsTestDirection b →
      tsupport a.val ⊆ Metric.ball x ε → tsupport b.val ⊆ Metric.ball x ε → Kills β a b) :
    ∀ᵐ z ∂(MeasureTheory.volume : Measure T3), z ∈ Metric.ball x ε → β z = 0 := ...
```

```
-- FrohmanianTether/TetherWeightProof.lean, line 406
public theorem tetherKernel_aeEnstrophyWeight : AEEnstrophyWeight TetherKernel := ...
```

```
-- FrohmanianTether/TetherWeightProof.lean, line 422
public theorem aeEnstrophyWeight_nontrivial :
    HasTetherKernelDensity TetherKernel canonicalTetherDensity ∧
    (∀ ω, IsRegularVorticity ω →
      LocallyIntegrable (canonicalTetherDensity ω) (MeasureTheory.volume : Measure T3)) ∧
    IsRegularVorticity (rotOrbit 0) ∧ -kappa * Enstrophy (rotOrbit 0) ≠ 0 := ...
```

```
-- FrohmanianTether/TetherWeightProof.lean, line 436
public theorem tetherKernel_core_inputs :
    (∃ α, HasTetherKernelDensity TetherKernel α ∧
      ∀ ω, IsRegularVorticity ω → LocallyIntegrable (α ω) (MeasureTheory.volume : Measure T3))
      ↪  ∧
    AEEnstrophyWeight TetherKernel ∧ SaturatesTetherQuadratic TetherKernel := ...
```

A.4 Transported (C1)

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 52
@[expose] public def InvariantUnderTransportedCoadjointAction (SDiff : Type*) [Group SDiff]
    [MulAction SDiff CoadjointOrbit] (B : CoadjointOrbit → Functional → Functional → ℝ) : Prop
    ↪  :=
  ∀ (g : SDiff) (F G : Functional) (ω : CoadjointOrbit),
    B (CoadjointAction g ω) (fun η => F (g⁻¹ • η)) (fun η => G (g⁻¹ • η)) = B ω F G
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 82
@[expose] public def transSubgroup : Subgroup G where
  carrier := {g | ∃ h, g = translate h}
  one_mem' := ⟨0, by
    apply Subtype.ext; apply Equiv.ext; intro x; simp [translate]⟩
  mul_mem' := by
    rintro _ _ ⟨h, rfl⟩ ⟨k, rfl⟩
    refine ⟨k + h, ?_⟩
```

```
    apply Subtype.ext; apply Equiv.ext; intro x
    simp [translate, add_assoc]
  inv_mem' := by
    rintro _ ⟨h, rfl⟩
    exact ⟨-h, translate_inv h⟩
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 70
public theorem translate_smul_val (h : T3) (ω : CoadjointOrbit) :
    (translate h • ω).val = fun y => ω.val (y - h) := ...
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 118
@[expose] public def l2Translate (h : T3) : L2Field →ₗᵢ[ℝ] L2Field :=
  Lp.compMeasurePreservingₗᵢ ℝ (fun x : T3 => x - h) (mp_sub h)
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 136
public theorem l2Translate_mem_divFreeL2 (h : T3) {v : L2Field} (hv : v ∈ DivFreeL2) :
    l2Translate h v ∈ DivFreeL2 := ...
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 155
public theorem isFunctionalGradient_translate (h : T3) {F : CoadjointOrbit → ℝ}
    {ω : CoadjointOrbit} {g : L2Field} (hg : IsFunctionalGradient F ω g) :
    IsFunctionalGradient (fun η => F ((translate h)⁻¹ • η)) (translate h • ω)
      (l2Translate h g) := ...
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 181
public theorem functionalDerivative_translate (h : T3) (F : CoadjointOrbit → ℝ)
    (ω : CoadjointOrbit) :
    FunctionalDerivative (fun η => F ((translate h)⁻¹ • η)) (translate h • ω)
      =ᵐ[MeasureTheory.volume] fun x => FunctionalDerivative F ω (x - h) := ...
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 211
public theorem biotSavart_translate (h : T3) (ω : CoadjointOrbit) :
    velocity_from_vorticity (translate h • ω) = fun x => velocity_from_vorticity ω (x - h) :=
    ↪  ...
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 225
public theorem tI_translate (h : T3) (ω : CoadjointOrbit)
    {a b a' b' : T3 → EuclideanSpace ℝ (Fin 3)}
    (ha : a' =ᵐ[MeasureTheory.volume] fun x => a (x - h))
    (hb : b' =ᵐ[MeasureTheory.volume] fun x => b (x - h)) :
    tI (translate h • ω) a' b' = tI ω a b := ...
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 259
public theorem tetherKernel_translate (h : T3) (F G : Functional) (ω : CoadjointOrbit) :
    TetherKernel (translate h • ω) (fun η => F ((translate h)⁻¹ • η))
      (fun η => G ((translate h)⁻¹ • η)) = TetherKernel ω F G := ...
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 272
public theorem tetherKernel_transported_invariant :
    InvariantUnderTransportedCoadjointAction transSubgroup TetherKernel := ...
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 317
public theorem not_tetherKernel_invariant_transSubgroup :
    ¬ InvariantUnderCoadjointAction transSubgroup TetherKernel := ...
```

```
-- FrohmanianTether/TetherEquivariantC1.lean, line 327
public theorem not_transported_invariant_pointEval :
    ¬ InvariantUnderTransportedCoadjointAction transSubgroup pointEvalKernel := ...
```

```
-- FrohmanianTether/PlanSteps.lean, line 76
theorem tether_bound_not_scale_invariant (S C κ M : ℝ) (hκ : 0 < κ) (hM : M ≠ 0) :
    ¬ ∀ l : ℝ, 0 < l → l ^ 6 * S ≤ C * (l ^ 2 * M) ^ 3 - κ * (l ^ 2 * M) ^ 4 := ...
```

A.5 Refutations and degeneracy of the encoding

```
-- FrohmanianTether/ConcreteSDiff.lean, line 112
theorem not_transitive_on_casimir_levels :
    ¬ ∀ ω₁ ω₂ : CoadjointOrbit,
      (∀ f : ℝ → ℝ, Casimir f ω₁ = Casimir f ω₂) → ∃ g : G, g • ω₁ = ω₂ := ...
```

```
-- FrohmanianTether/StrainBoundRefutation.lean, line 78
public theorem not_forall_strain_le_sup_vorticity (C : ℝ) :
    ¬ ∀ (u : VelocityField) (x : T3),
      ‖fderiv ℝ u x‖ ≤ C * vorticity_sup_norm (curl u) := ...
```

```
-- FrohmanianTether/Uniqueness.lean, line 335
public theorem antisymmetric_clause_forces_zero
    (B : CoadjointOrbit → Functional → Functional → ℝ)
    (h_antisym : ∀ ω F G, B ω F G = -B ω G F)
    (h_sat : SaturatesTetherQuadratic B)
    (h_polarB : Polarizes B)
    (h_polarT : Polarizes TetherKernel) :
    ∀ (ω : CoadjointOrbit) (F G : Functional), B ω F G = 0 ∧ TetherKernel ω F G = 0 := ...
```

```
-- FrohmanianTether/ArnoldGeometric.lean, line 407
public theorem functionalDerivative_indep_of_not_differentiable
    {ω : CoadjointOrbit} (hω : ¬ ∀ i x, DifferentiableAt ℝ (fun y => ω.val y i) x)
    (F G : CoadjointOrbit → ℝ) :
    FunctionalDerivative F ω = FunctionalDerivative G ω := ...
```

```
-- FrohmanianTether/Uniqueness.lean, line 317
public theorem polarizes_tetherKernel_zero_of_not_differentiable
    (hP : Polarizes TetherKernel)
    {ω : CoadjointOrbit} (hω : ¬ ∀ i x, DifferentiableAt ℝ (fun y => ω.val y i) x)
    (F G : Functional) : TetherKernel ω F G = 0 := ...
```

```
-- FrohmanianTether/TetherNondegeneracy.lean, line 390
public theorem not_tetherKernel_polarizes :
    ¬ ∀ ω F G, 2 * TetherKernel ω F G = TetherKernel ω (fun η => F η + G η) (fun η => F η + G
    ↪  η) - TetherKernel ω F F - TetherKernel ω G G := ...
```

```
-- FrohmanianTether/Uniqueness.lean, line 481
public theorem not_pointwiseEnstrophyWeight_of_density
    (B : CoadjointOrbit → Functional → Functional → ℝ)
    (hex : ∃ α, HasTetherKernelDensity B α) : ¬ PointwiseEnstrophyWeight B := ...
```

```
-- FrohmanianTether/Uniqueness.lean, line 394
public theorem not_step1Hyp (B : CoadjointOrbit → Functional → Functional → ℝ) :
    ¬ Step1Hyp B := ...
```

```
-- FrohmanianTether/TetherWeight.lean, line 178
public theorem integral_indicator_vitaliSet_mul (f : T3 → ℝ) :
    ∫ x, vitaliSet.indicator (fun _ => (1 : ℝ)) x * f x ∂(MeasureTheory.volume : Measure T3) =
    ↪  0 := ...
```

```
-- FrohmanianTether/TetherWeight.lean, line 223
public theorem not_aeEnstrophyWeightUnrestricted_tetherKernel :
    ¬ AEEnstrophyWeightUnrestricted TetherKernel := ...
```

```
-- FrohmanianTether/TetherWeight.lean, line 237
public theorem not_aeEnstrophyWeightUnrestricted_of_eq_tether
    (B : CoadjointOrbit → Functional → Functional → ℝ)
    (hB : ∀ ω F G, B ω F G = TetherKernel ω F G) : ¬ AEEnstrophyWeightUnrestricted B := ...
```

```
-- FrohmanianTether/TetherC1.lean, line 56
@[expose] public def translate (h : T3) : G := ⟨Equiv.addRight h, good_addRight h⟩
```

```
-- FrohmanianTether/TetherC1.lean, line 191
public theorem not_tetherKernel_invariant_concrete :
    ¬ InvariantUnderCoadjointAction ConcreteSDiff.G TetherKernel := ...
```

```
-- FrohmanianTether/TetherNondegeneracy.lean, line 419
public theorem not_forall_functionalDerivative_sub :
    ¬ ∀ ω (F F' : CoadjointOrbit → ℝ), FunctionalDerivative (fun η => F η - F' η) ω =
      FunctionalDerivative F ω - FunctionalDerivative F' ω := ...
```

A.6 Analytic layer

```
-- FrohmanianTether/AnalyticPipeline.lean, line 85
public theorem comparison_ode_stability
    (C κ'' : ℝ) (hC : 0 < C) (hκ : 0 < κ'')
    (y : ℝ → ℝ)
    (hdiff : ∀ s ≥ (0 : ℝ), DifferentiableAt ℝ y s)
    (hy_dot : ∀ s ≥ (0 : ℝ), deriv y s ≤ C * (y s) ^ 2 - κ'' * (y s) ^ 3)
    (t : ℝ) (ht : 0 ≤ t) :
    y t ≤ max (y 0) (C / κ'') := ...
```

```
-- FrohmanianTether/TetheredLyapunov.lean, line 1480
public theorem comparison_ode_stability_Icc
    (C κ'' : ℝ) (hC : 0 < C) (hκ : 0 < κ'') (y : ℝ → ℝ) (T : ℝ)
    (hdiff : ∀ s ∈ Set.Icc (0 : ℝ) T, DifferentiableAt ℝ y s)
    (hy_dot : ∀ s ∈ Set.Icc (0 : ℝ) T, deriv y s ≤ C * (y s) ^ 2 - κ'' * (y s) ^ 3)
    (t : ℝ) (ht : t ∈ Set.Icc (0 : ℝ) T) :
    y t ≤ max (y 0) (C / κ'') := ...
```

```
-- FrohmanianTether/TetheredLyapunov.lean, line 1661
public theorem riccati_ceiling_of_vorticity_di
    (u : TimeDependentVelocity) (C κ'' : ℝ)
```

```
    (hC : 0 < C) (hκ : 0 < κ'')
    (hdiff : ∀ s ≥ (0 : ℝ),
      DifferentiableAt ℝ (fun τ => vorticity_sup_norm (vorticity (u τ))) s)
    (hDI : ∀ s ≥ (0 : ℝ),
      deriv (fun τ => vorticity_sup_norm (vorticity (u τ))) s ≤
        C * vorticity_sup_norm (vorticity (u s)) ^ 2 -
          κ'' * vorticity_sup_norm (vorticity (u s)) ^ 3)
    (hcont : Continuous fun τ : ℝ => vorticity_sup_norm (vorticity (u τ))) :
    ∃ Y : ℝ,
      (∀ τ ≥ (0 : ℝ), vorticity_sup_norm (vorticity (u τ)) ≤ Y) ∧
      Continuous (fun τ : ℝ => vorticity_sup_norm (vorticity (u τ))) := ...
```

```
-- FrohmanianTether/TetheredLyapunov.lean, line 1511
@[expose] public def LocalTetherRate (u₀ : VelocityField) (ν : ℝ) : Prop :=
  ∃ C κ'' : ℝ, 0 < C ∧ 0 < κ'' ∧
    ∀ (T : ℝ) (u : TimeDependentVelocity) (p : TimeDependentPressure),
      IsLocalPhysicalSolution ν T u₀ u p →
        (∀ s ∈ Set.Icc (0 : ℝ) T,
          DifferentiableAt ℝ (fun τ => vorticity_sup_norm (vorticity (u τ))) s) ∧
        (∀ s ∈ Set.Icc (0 : ℝ) T,
          deriv (fun τ => vorticity_sup_norm (vorticity (u τ))) s ≤
            C * vorticity_sup_norm (vorticity (u s)) ^ 2 -
              κ'' * vorticity_sup_norm (vorticity (u s)) ^ 3)
```

```
-- FrohmanianTether/TetheredLyapunov.lean, line 1459
@[expose] public def LocalTetherBound (u₀ : VelocityField) (ν : ℝ) : Prop :=
  ∃ Y : ℝ, ∀ (T : ℝ) (u : TimeDependentVelocity) (p : TimeDependentPressure),
    IsLocalPhysicalSolution ν T u₀ u p →
      ∀ t ∈ Set.Icc (0 : ℝ) T, vorticity_sup_norm (vorticity (u t)) ≤ Y
```

```
-- FrohmanianTether/TetheredLyapunov.lean, line 1467
@[expose] public def LocalFiniteTimeVorticityBound (u₀ : VelocityField) (ν : ℝ) : Prop :=
  ∀ T₀ : ℝ, ∃ C : ℝ, ∀ (T : ℝ) (u : TimeDependentVelocity) (p : TimeDependentPressure),
    T ≤ T₀ → IsLocalPhysicalSolution ν T u₀ u p →
      ∀ t ∈ Set.Icc (0 : ℝ) T, vorticity_sup_norm (vorticity (u t)) ≤ C
```

```
-- FrohmanianTether/ClayStatements.lean, line 57
@[expose] public def LocalTetherClosingBound (ν : ℝ) : Prop :=
  ∀ u₀ : VelocityField,
    (∀ x, div u₀ x = 0) → ContDiff ℝ (⊤ : ℕ∞) u₀ →
      Integrable (fun x : T3 => ‖u₀ x‖ ^ 2) →
        TetheredLyapunov.LocalTetherBound u₀ ν
```

```
-- FrohmanianTether/TetheredLyapunov.lean, line 1541
@[expose] public def BKMGlobalContinuation (u₀ : VelocityField) (ν : ℝ) : Prop :=
  Nonempty (KatoLocalWitness u₀ ν) → LocalFiniteTimeVorticityBound u₀ ν →
    ∃ (u : TimeDependentVelocity) (p : TimeDependentPressure),
      u 0 = u₀ ∧ NS_PDE u p ν ∧ (∀ t ≥ (0 : ℝ), ContDiff ℝ (⊤ : ℕ∞) (u t)) ∧
        IsPhysicalNSSolution u
```

```
-- FrohmanianTether/TetheredLyapunov.lean, line 1549
@[expose] public def SmoothPhysicalUniqueness (u₀ : VelocityField) (ν : ℝ) : Prop :=
  ∀ (T : ℝ) (u v : TimeDependentVelocity) (p q : TimeDependentPressure),
    IsLocalPhysicalSolution ν T u₀ u p → IsLocalPhysicalSolution ν T u₀ v q →
      ∀ t ∈ Set.Icc (0 : ℝ) T, u t = v t
```

```
-- FrohmanianTether/SymplecticTether.lean, line 403
@[expose] public abbrev FrohmanianTetherRate (u₀ : VelocityField) (ν : ℝ) : Prop :=
  ∀ u : ℝ → VelocityField, ∀ p : ℝ → PressureField,
    0 < ν → NS_PDE u p ν → u 0 = u₀ → IsPhysicalNSSolution u →
      ∃ C κ'' : ℝ, 0 < C ∧ 0 < κ'' ∧
        (∀ s ≥ (0 : ℝ),
          DifferentiableAt ℝ (fun τ => vorticity_sup_norm (vorticity (u τ))) s) ∧
        (∀ s ≥ (0 : ℝ),
          deriv (fun τ => vorticity_sup_norm (vorticity (u τ))) s ≤
            C * vorticity_sup_norm (vorticity (u s)) ^ 2 -
              κ'' * vorticity_sup_norm (vorticity (u s)) ^ 3) ∧
        Continuous (fun τ : ℝ => vorticity_sup_norm (vorticity (u τ))) ∧
        (∀ τ ≥ (0 : ℝ), BddAbove (Set.range fun x => ‖vorticity (u τ) x‖))
```

```
-- FrohmanianTether/SymplecticTether.lean, line 391
@[expose] public abbrev FrohmanianTetherBound (u₀ : VelocityField) (ν : ℝ) : Prop :=
  ∀ u : ℝ → VelocityField, ∀ p : ℝ → PressureField,
    NS_PDE u p ν → u 0 = u₀ → IsPhysicalNSSolution u →
      ∃ Y : ℝ,
        (∀ τ ≥ (0 : ℝ), vorticity_sup_norm (vorticity (u τ)) ≤ Y) ∧
        Continuous (fun τ : ℝ => vorticity_sup_norm (vorticity (u τ))) ∧
        (∀ τ ≥ (0 : ℝ), BddAbove (Set.range fun x => ‖vorticity (u τ) x‖))
```

```
-- FrohmanianTether/ClayStatements.lean, line 48
@[expose] public def FrohmanianTetherClosingBound (ν : ℝ) : Prop :=
  ∀ u₀ : VelocityField,
    (∀ x, div u₀ x = 0) → ContDiff ℝ (⊤ : ℕ∞) u₀ →
      Integrable (fun x : T3 => ‖u₀ x‖ ^ 2) →
        FrohmanianTetherBound u₀ ν
```

```
-- FrohmanianTether/TetheredLyapunov.lean, line 1427
@[expose] public def KatoUniquenessOnExistenceTimes (u₀ : VelocityField) (ν : ℝ)
    (w : KatoLocalWitness u₀ ν) : Prop :=
  ∀ (T' : ℝ) (v : TimeDependentVelocity) (q : TimeDependentPressure),
    T' ∈ existenceTimes u₀ ν → v 0 = u₀ → NS_PDE_On v q ν T' →
    (∀ t ∈ Set.Icc (0 : ℝ) T', ContDiff ℝ (⊤ : ℕ∞) (v t)) →
    IsPhysicalNSSolutionOn v T' →
    ∀ t ∈ Set.Icc (0 : ℝ) T', w.u t = v t
```

```
-- FrohmanianTether/TetheredLyapunov.lean, line 1582
public theorem conditional_regularity
    (u₀ : VelocityField) (ν : ℝ)
    (w : KatoLocalWitness u₀ ν)
    (hcont : BKMGlobalContinuation u₀ ν)
    (hRiccati : LocalTetherBound u₀ ν) :
  ∃ (u : ℝ → VelocityField) (p : ℝ → PressureField),
    NS_PDE u p ν ∧
    u 0 = u₀ ∧
    (∀ t ≥ (0 : ℝ), ContDiff ℝ (⊤ : ℕ∞) (u t)) ∧
    IsPhysicalNSSolution u ∧
    (∀ t ≥ (0 : ℝ), vorticity_sup_norm (vorticity (u t)) ≥ 0) ∧
    (∀ t ≥ (0 : ℝ), ∀ x, div (u t) x = 0) := ...
```

```
-- FrohmanianTether/ClosureBlueprint.lean, line 201
public theorem conditional_regularity_of_open_hyps
    (u₀ : VelocityField) (ν : ℝ)
    (w : KatoLocalWitness u₀ ν)
```

```
    (hcont : TetheredLyapunov.BKMGlobalContinuation u₀ ν)
    (hRiccati : TetheredLyapunov.LocalTetherBound u₀ ν) :
    ∃ (u : ℝ → VelocityField) (p : ℝ → PressureField),
      NS_PDE u p ν ∧
      u 0 = u₀ ∧
      (∀ t ≥ (0 : ℝ), ContDiff ℝ (⊤ : ℕ∞) (u t)) ∧
      (∀ t ≥ (0 : ℝ), vorticity_sup_norm (vorticity (u t)) ≥ 0) ∧
      (∀ t ≥ (0 : ℝ), ∀ x, div (u t) x = 0) := ...
```

```
-- FrohmanianTether/ConditionalRegularity.lean, line 124
public theorem conditional_regularity_of_open_hyps
    (u0 : TimeDependentVelocity)
    (_h_div_free : ∀ t x, div (u0 t) x = 0)
    (_h_smooth : IsSmooth (u0 0))
    (_h_finite_energy : Integrable (fun x : T3 => ‖u0 0 x‖ ^ 2))
    (ν : ℝ) (_h_ν_pos : ν > 0)
    (w : KatoLocalWitness (u0 0) ν)
    (hcont : TetheredLyapunov.BKMGlobalContinuation (u0 0) ν)
    (hRiccati : TetheredLyapunov.LocalTetherBound (u0 0) ν)
    (huniq : TetheredLyapunov.SmoothPhysicalUniqueness (u0 0) ν) :
  ∃ (u : TimeDependentVelocity),
    (∀ t ≥ (0 : ℝ), IsSmooth (u t)) ∧
    (u 0 = u0 0) ∧
    (∀ t ≥ (0 : ℝ), satisfies_NavierStokes u ν) ∧
    (∀ t ≥ (0 : ℝ), vorticity_sup_norm (vorticity (u t)) ≥ 0) ∧
    (∀ v : TimeDependentVelocity,
      (∀ t ≥ (0 : ℝ), IsSmooth (v t)) →
      v 0 = u0 0 →
      (∀ t ≥ (0 : ℝ), satisfies_NavierStokes v ν) →
      ∀ t ≥ (0 : ℝ), v t = u t) := ...
```

```
-- FrohmanianTether/ConditionalRegularity.lean, line 60
public theorem frohmanian_tether_theorem
    (hKato : ∀ u₀ ν, ContDiff ℝ (⊤ : ℕ∞) u₀ → (∀ x, div u₀ x = 0) →
      Integrable (fun x : T3 => ‖u₀ x‖ ^ 2) → 0 < ν →
      KatoLocalWitness u₀ ν)
    (hcont : ∀ u₀ ν, TetheredLyapunov.BKMGlobalContinuation u₀ ν)
    (hDI : ∀ u₀ ν, TetheredLyapunov.LocalTetherRate u₀ ν) :
  ∃ (𝔗_F : CoadjointOrbit → Functional → Functional → ℝ),
    (∀ (B : CoadjointOrbit → Functional → Functional → ℝ)
      (α : CoadjointOrbit → T3 → ℝ),
      HasTetherKernelDensity B α →
      (∀ ω x, α ω x = canonicalTetherDensity ω x) →
      ∀ ω F G, B ω F G = 𝔗_F ω F G) ∧
    (∀ (u₀ : VelocityField) (ν : ℝ),
      0 < ν →
      (∀ x, div u₀ x = 0) →
      ContDiff ℝ (⊤ : ℕ∞) u₀ →
      Integrable (fun x : T3 => ‖u₀ x‖ ^ 2) →
      ∃ (u : ℝ → VelocityField) (p : ℝ → PressureField),
        NS_PDE u p ν ∧
        u 0 = u₀ ∧
        (∀ t ≥ (0 : ℝ), ContDiff ℝ (⊤ : ℕ∞) (u t)) ∧
        (∀ t ≥ (0 : ℝ), vorticity_sup_norm (vorticity (u t)) ≥ 0) ∧
        (∀ t ≥ (0 : ℝ), ∀ x, div (u t) x = 0)) := ...
```

```
-- FrohmanianTether/ClayStatements.lean, line 497
public theorem conditional_regularity_nonvacuous_at_zero (ν : ℝ) (hν : 0 < ν) :
    ∃ (u : TimeDependentVelocity),
      (∀ t ≥ (0 : ℝ), ConditionalRegularity.IsSmooth (u t)) ∧
```

```
      (u 0 = 0) ∧
      (∀ t ≥ (0 : ℝ), ConditionalRegularity.satisfies_NavierStokes u ν) ∧
      (∀ t ≥ (0 : ℝ), ConditionalRegularity.vorticity_sup_norm (vorticity (u t)) ≥ 0) ∧
      (∀ v : TimeDependentVelocity,
        (∀ t ≥ (0 : ℝ), ConditionalRegularity.IsSmooth (v t)) →
        v 0 = 0 →
        (∀ t ≥ (0 : ℝ), ConditionalRegularity.satisfies_NavierStokes v ν) →
        ∀ t ≥ (0 : ℝ), v t = u t) := ...
```

```
-- FrohmanianTether/ClayStatements.lean, line 403
public theorem stepField_not_physical : ¬ IsPhysicalNSSolution stepField := ...
```

A.7 Periodic Clay statement

```
-- FrohmanianTether/ClayStatements.lean, line 208
@[expose] public def IsZ3Periodic {α : Type*} (f : T3 → α) : Prop :=
  ∀ (k : Fin 3 → ℤ) (x : T3), f (x + latticeVec k) = f x
```

```
-- FrohmanianTether/ClayStatements.lean, line 247
@[expose] public def IsPeriodicSmoothSolution (ν : ℝ) (u₀ : VelocityField)
    (f : ℝ → T3 → EuclideanSpace ℝ (Fin 3))
    (u : TimeDependentVelocity) (p : TimeDependentPressure) : Prop :=
  ContDiffOn ℝ ∞ (fun q : ℝ × T3 => u q.1 q.2) (Set.Ici (0 : ℝ) ×ˢ Set.univ) ∧
  ContDiffOn ℝ ∞ (fun q : ℝ × T3 => p q.1 q.2) (Set.Ici (0 : ℝ) ×ˢ Set.univ) ∧
  navier_stokes_eq_Ici u p ν f ∧
  u 0 = u₀ ∧
  ∀ t ≥ (0 : ℝ), IsZ3Periodic (u t) ∧ IsZ3Periodic (p t)
```

```
-- FrohmanianTether/ClayStatements.lean, line 529
@[expose] public def IsPeriodicSmoothSolutionOn (ν T : ℝ) (u₀ : VelocityField)
    (u : TimeDependentVelocity) (p : TimeDependentPressure) : Prop :=
  ContDiffOn ℝ ∞ (fun q : ℝ × T3 => u q.1 q.2) (Set.Ico (0 : ℝ) T ×ˢ Set.univ) ∧
  ContDiffOn ℝ ∞ (fun q : ℝ × T3 => p q.1 q.2) (Set.Ico (0 : ℝ) T ×ˢ Set.univ) ∧
  (∀ t ∈ Set.Ico (0 : ℝ) T, ∀ x : T3,
    derivWithin (fun s => u s x) (Set.Ici 0) t + convective (u t) (u t) x +
        pressureGradient (p t) x = ν • laplacian (u t) x ∧ div (u t) x = 0) ∧
  u 0 = u₀ ∧
  ∀ t ∈ Set.Ico (0 : ℝ) T, IsZ3Periodic (u t) ∧ IsZ3Periodic (p t)
```

```
-- FrohmanianTether/ClayStatements.lean, line 257
@[expose] public def Clay_B_exact : Prop :=
  ∀ (ν : ℝ) (u₀ : VelocityField),
    0 < ν →
    ContDiff ℝ (⊤ : ℕ∞) u₀ →
    (∀ x, div u₀ x = 0) →
    IsZ3Periodic u₀ →
    ∃ (u : TimeDependentVelocity) (p : TimeDependentPressure),
      IsPeriodicSmoothSolution ν u₀ (fun _ _ => 0) u p
```

```
-- FrohmanianTether/ClayStatements.lean, line 545
@[expose] public def PeriodicTetherClosingBound (ν : ℝ) : Prop :=
  ∀ u₀ : VelocityField,
    (∀ x, div u₀ x = 0) → ContDiff ℝ (⊤ : ℕ∞) u₀ → IsZ3Periodic u₀ →
      ∃ Y : ℝ, ∀ (T : ℝ) (u : TimeDependentVelocity) (p : TimeDependentPressure),
        IsPeriodicSmoothSolutionOn ν T u₀ u p →
          ∀ t ∈ Set.Ico (0 : ℝ) T, vorticity_sup_norm (vorticity (u t)) ≤ Y
```

```
-- FrohmanianTether/ClayStatements.lean, line 557
@[expose] public def PeriodicBKMContinuation (ν : ℝ) : Prop :=
  ∀ u₀ : VelocityField,
    (∀ x, div u₀ x = 0) → ContDiff ℝ (⊤ : ℕ∞) u₀ → IsZ3Periodic u₀ →
      (∃ Y : ℝ, ∀ (T : ℝ) (u : TimeDependentVelocity) (p : TimeDependentPressure),
        IsPeriodicSmoothSolutionOn ν T u₀ u p →
          ∀ t ∈ Set.Ico (0 : ℝ) T, vorticity_sup_norm (vorticity (u t)) ≤ Y) →
        ∃ (u : TimeDependentVelocity) (p : TimeDependentPressure),
          IsPeriodicSmoothSolution ν u₀ (fun _ _ => 0) u p
```

```
-- FrohmanianTether/ClayStatements.lean, line 569
public theorem clay_B_exact_of_periodic_tether_bound
    (hclose : ∀ ν > (0 : ℝ), PeriodicTetherClosingBound ν)
    (hBKM : ∀ ν > (0 : ℝ), PeriodicBKMContinuation ν) : Clay_B_exact := ...
```

A.8 Explicit exact solutions

```
-- FrohmanianTether/ExplicitFlows.lean, line 49
@[expose] public def tp (k α β : ℝ) (j l : Fin 3) (y : T3) : ℝ :=
  α * Real.sin (k * y j) + β * Real.cos (k * y l)
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 130
@[expose] public def trigField (k : ℝ) (α β : Fin 3 → ℝ) (j l : Fin 3 → Fin 3) : VelocityField
↪  :=
  fun y => WithLp.toLp 2 (fun i => tp k (α i) (β i) (j i) (l i) y)
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 184
@[expose] public def decay (c t : ℝ) : ℝ := if 0 ≤ t then Real.exp (c * t) else 1 + c * t
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 208
public theorem hasDerivAt_decay (c : ℝ) {t : ℝ} (ht : 0 ≤ t) :
    HasDerivAt (decay c) (c * decay c t) t := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 158
public theorem laplacian_trigField (k : ℝ) (α β : Fin 3 → ℝ) (j l : Fin 3 → Fin 3) (x : T3) :
    laplacian (trigField k α β j l) x = (-k ^ 2) • trigField k α β j l x := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 233
@[expose] public def flow (ν k : ℝ) (α β : Fin 3 → ℝ) (j l : Fin 3 → Fin 3) :
    TimeDependentVelocity :=
  fun t y => decay (-(ν * k ^ 2)) t • trigField k α β j l y
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 248
@[expose] public def flowPressure (ν k : ℝ) (p₀ : PressureField) : TimeDependentPressure :=
  fun t y => decay (-(ν * k ^ 2)) t ^ 2 * p₀ y
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 270
public theorem flow_navier_stokes (ν k : ℝ) (α β : Fin 3 → ℝ) (j l : Fin 3 → Fin 3)
    (p₀ : PressureField) (hp₀ : Differentiable ℝ p₀)
    (hdiv : ∀ x, div (trigField k α β j l) x = 0)
    (hbal : ∀ x, convective (trigField k α β j l) (trigField k α β j l) x +
      pressureGradient p₀ x = 0) :
    navier_stokes_eq (flow ν k α β j l) (flowPressure ν k p₀) ν := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 313
@[expose] public def shearProfile (k a : ℝ) : VelocityField :=
  trigField k (shearα a) 0 (fun _ => 2) (fun _ => 2)
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 317
@[expose] public def shearFlow (ν k a : ℝ) : TimeDependentVelocity :=
  flow ν k (shearα a) 0 (fun _ => 2) (fun _ => 2)
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 321
@[expose] public def shearPressure : TimeDependentPressure := fun _ _ => 0
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 328
public theorem shearFlow_apply (ν k a : ℝ) {t : ℝ} (ht : 0 ≤ t) (y : T3) :
    shearFlow ν k a t y =
      WithLp.toLp 2 ![Real.exp (-(ν * k ^ 2) * t) * a * Real.sin (k * y 2), 0, 0] := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 347
public theorem shearFlow_navier_stokes (ν k a : ℝ) :
    navier_stokes_eq (shearFlow ν k a) shearPressure ν := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 363
public theorem shearFlow_NS_PDE_On (ν k a T : ℝ) :
    NS_PDE_On (shearFlow ν k a) shearPressure ν T := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 594
public theorem shearFlow_ne_zero (ν : ℝ) {k a : ℝ} (hk : k ≠ 0) (ha : a ≠ 0) {t : ℝ}
    (ht : 0 ≤ t) : shearFlow ν k a t ≠ 0 := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 377
@[expose] public def abcProfile (k A B C : ℝ) : VelocityField :=
  trigField k (abcα A B C) (abcβ A B C) abcj abcl
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 384
public theorem abcProfile_apply (k A B C : ℝ) (y : T3) :
    abcProfile k A B C y = WithLp.toLp 2
      ![A * Real.sin (k * y 2) + C * Real.cos (k * y 1),
        B * Real.sin (k * y 0) + A * Real.cos (k * y 2),
        C * Real.sin (k * y 1) + B * Real.cos (k * y 0)] := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 381
@[expose] public def abcFlow (ν k A B C : ℝ) : TimeDependentVelocity :=
  flow ν k (abcα A B C) (abcβ A B C) abcj abcl
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 393
@[expose] public def abcPressure₀ (k A B C : ℝ) : PressureField := fun y =>
  -((abcProfile k A B C y 0) ^ 2 + (abcProfile k A B C y 1) ^ 2 +
    (abcProfile k A B C y 2) ^ 2) / 2
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 397
public theorem abcPressure₀_eq_norm (k A B C : ℝ) (y : T3) :
    abcPressure₀ k A B C y = -‖abcProfile k A B C y‖ ^ 2 / 2 := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 403
@[expose] public def abcPressure (ν k A B C : ℝ) : TimeDependentPressure :=
  flowPressure ν k (abcPressure₀ k A B C)
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 437
public theorem abc_balance (k A B C : ℝ) (x : T3) :
    convective (abcProfile k A B C) (abcProfile k A B C) x +
      pressureGradient (abcPressure₀ k A B C) x = 0 := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 450
public theorem abcFlow_navier_stokes (ν k A B C : ℝ) :
    navier_stokes_eq (abcFlow ν k A B C) (abcPressure ν k A B C) ν := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 459
public theorem abcFlow_NS_PDE_On (ν k A B C T : ℝ) :
    NS_PDE_On (abcFlow ν k A B C) (abcPressure ν k A B C) ν T := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 629
public theorem curl_abcProfile (k A B C : ℝ) (x : T3) :
    curl (abcProfile k A B C) x = k • abcProfile k A B C x := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 605
public theorem abcFlow_apply_zero (ν k A B C : ℝ) {t : ℝ} (ht : 0 ≤ t) :
    abcFlow ν k A B C t 0 = Real.exp (-(ν * k ^ 2) * t) • WithLp.toLp 2 ![C, A, B] := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 612
public theorem abcFlow_ne_zero (ν k : ℝ) {A B C : ℝ} (hABC : A ≠ 0 ∨ B ≠ 0 ∨ C ≠ 0) {t : ℝ}
    (ht : 0 ≤ t) : abcFlow ν k A B C t ≠ 0 := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 465
public theorem flow_smooth (ν k : ℝ) (α β : Fin 3 → ℝ) (j l : Fin 3 → Fin 3) (t : ℝ) :
    ContDiff ℝ (⊤ : ℕ∞) (flow ν k α β j l t) := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 503
public theorem flow_physical_clauses (ν k : ℝ) (α β : Fin 3 → ℝ) (j l : Fin 3 → Fin 3) (T : ℝ)
↪  :
    (∀ t ∈ Set.Icc (0 : ℝ) T, ∀ x : T3, DifferentiableAt ℝ (fun s => flow ν k α β j l s x) t) ∧
    (∀ t ∈ Set.Icc (0 : ℝ) T, Continuous (flow ν k α β j l t)) ∧
    (∃ v₀ : VelocityField, ∀ t ≤ (0 : ℝ),
      flow ν k α β j l t = flow ν k α β j l 0 + t • v₀) := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 557
public theorem shearFlow_isPeriodicSmoothSolution (ν : ℝ) (m : ℤ) (a : ℝ) :
    FrohmanianTether.ClayStatements.IsPeriodicSmoothSolution ν
      (shearProfile (2 * Real.pi * m) a) (fun _ _ => 0)
      (shearFlow ν (2 * Real.pi * m) a) shearPressure := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 585
public theorem abcFlow_isPeriodicSmoothSolution (ν : ℝ) (m : ℤ) (A B C : ℝ) :
    FrohmanianTether.ClayStatements.IsPeriodicSmoothSolution ν
      (abcProfile (2 * Real.pi * m) A B C) (fun _ _ => 0)
      (abcFlow ν (2 * Real.pi * m) A B C) (abcPressure ν (2 * Real.pi * m) A B C) := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 806
public theorem clay_B_instance_shear (ν : ℝ) (m : ℤ) (a : ℝ) :
    ContDiff ℝ (⊤ : ℕ∞) (shearProfile (2 * Real.pi * m) a) ∧
    (∀ x, div (shearProfile (2 * Real.pi * m) a) x = 0) ∧
    FrohmanianTether.ClayStatements.IsZ3Periodic (shearProfile (2 * Real.pi * m) a) ∧
    ∃ (u : TimeDependentVelocity) (p : TimeDependentPressure),
      FrohmanianTether.ClayStatements.IsPeriodicSmoothSolution ν
        (shearProfile (2 * Real.pi * m) a) (fun _ _ => 0) u p := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 818
public theorem clay_B_instance_abc (ν : ℝ) (m : ℤ) (A B C : ℝ) :
    ContDiff ℝ (⊤ : ℕ∞) (abcProfile (2 * Real.pi * m) A B C) ∧
    (∀ x, div (abcProfile (2 * Real.pi * m) A B C) x = 0) ∧
    FrohmanianTether.ClayStatements.IsZ3Periodic (abcProfile (2 * Real.pi * m) A B C) ∧
    ∃ (u : TimeDependentVelocity) (p : TimeDependentPressure),
      FrohmanianTether.ClayStatements.IsPeriodicSmoothSolution ν
        (abcProfile (2 * Real.pi * m) A B C) (fun _ _ => 0) u p := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 830
public theorem shearFlow_isPeriodicSmoothSolutionOn (ν T : ℝ) (m : ℤ) (a : ℝ) :
    FrohmanianTether.ClayStatements.IsPeriodicSmoothSolutionOn ν T
      (shearProfile (2 * Real.pi * m) a) (shearFlow ν (2 * Real.pi * m) a) shearPressure := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 836
public theorem abcFlow_isPeriodicSmoothSolutionOn (ν T : ℝ) (m : ℤ) (A B C : ℝ) :
    FrohmanianTether.ClayStatements.IsPeriodicSmoothSolutionOn ν T
      (abcProfile (2 * Real.pi * m) A B C) (abcFlow ν (2 * Real.pi * m) A B C)
      (abcPressure ν (2 * Real.pi * m) A B C) := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 664
public theorem not_integrable_sq_of_periodic {u : VelocityField} (hc : Continuous u)
    (hp : FrohmanianTether.ClayStatements.IsZ3Periodic u) (hne : u ≠ 0) :
    ¬ Integrable (fun x => ‖u x‖ ^ 2) := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 735
public theorem isEmpty_katoLocalWitness_of_periodic {u₀ : VelocityField} {ν : ℝ}
    (hp : FrohmanianTether.ClayStatements.IsZ3Periodic u₀) (hne : u₀ ≠ 0) :
    IsEmpty (KatoLocalWitness u₀ ν) := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 756
public theorem isEmpty_katoLocalWitness_shear (ν : ℝ) {m : ℤ} (hm : m ≠ 0) {a : ℝ}
    (ha : a ≠ 0) : IsEmpty (KatoLocalWitness (shearProfile (2 * Real.pi * m) a) ν) := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 762
public theorem isEmpty_katoLocalWitness_abc (ν : ℝ) (m : ℤ) {A B C : ℝ}
    (hABC : A ≠ 0 ∨ B ≠ 0 ∨ C ≠ 0) :
    IsEmpty (KatoLocalWitness (abcProfile (2 * Real.pi * m) A B C) ν) := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 782
public theorem localTetherBound_vacuous_of_periodic {u₀ : VelocityField} {ν : ℝ}
    (hp : FrohmanianTether.ClayStatements.IsZ3Periodic u₀) (hne : u₀ ≠ 0) :
    TetheredLyapunov.LocalTetherBound u₀ ν := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 790
public theorem smoothPhysicalUniqueness_vacuous_of_periodic {u₀ : VelocityField} {ν : ℝ}
    (hp : FrohmanianTether.ClayStatements.IsZ3Periodic u₀) (hne : u₀ ≠ 0) :
    TetheredLyapunov.SmoothPhysicalUniqueness u₀ ν := ...
```

```
-- FrohmanianTether/ExplicitFlows.lean, line 797
public theorem bkmGlobalContinuation_vacuous_of_periodic {u₀ : VelocityField} {ν : ℝ}
    (hp : FrohmanianTether.ClayStatements.IsZ3Periodic u₀) (hne : u₀ ≠ 0) :
    TetheredLyapunov.BKMGlobalContinuation u₀ ν := ...
```

```
-- FrohmanianTether/SymplecticConstraint.lean, line 718
public theorem nonzero_smooth_ns_solution_exists (ν : ℝ) (c : T3) (hc : c ≠ 0) :
    ∃ (u : TimeDependentVelocity) (p : TimeDependentPressure),
      u 0 ≠ 0 ∧
      (∀ t ≥ (0 : ℝ), IsSmooth (u t)) ∧
      navier_stokes_eq u p ν := ...
```

B Verification certificate

Repository	https://github.com/BenFrohman/frohmanian-tether-sdiff (version 0.1.4, DOI (to be assigned on publication))
Verified commit	`VERIFIEDCOMMIT` (Lean sources; later commits change only documentation, the manuscript and metadata)
Lean toolchain	`leanprover/lean4:v4.30.0-rc1` (pinned in `lean-toolchain`)
Mathlib	`v4.30.0-rc1`, rev `0c154d67103f74be3a0f2c509f72ccbf5be9f2a7` (pinned in `lake-manifest.json`)
Build	`lake build`: 2795 jobs, success
Modules	33 (`FrohmanianTether` and 32 submodules)
Theorems audited	760 (670 stated in the sources, 90 auto-generated)
Declarations audited	1074 (theorems, definitions, structures, instances)
Axioms	only `propext`, `Classical.choice`, `Quot.sound`
`sorry` / `admit` / `native_decide`	none (`scripts/check_no_sorry.py`: `OK`)
Axiom declarations	none

The CI runs and the full per-theorem axiom list are recorded in `CERTIFICATE.md` in the repository.

Reproduction (one command). From a fresh clone:

```
git clone https://github.com/BenFrohman/frohmanian-tether-sdiff
cd frohmanian-tether-sdiff
bash scripts/verify.sh        # equivalently: make verify
```

`scripts/verify.sh` installs `elan` if needed (which installs the toolchain pinned in `lean-toolchain`), runs `lake exe cache get` (Mathlib cache for the revision pinned in `lake-manifest.json`), `lake build`, `python3 scripts/check_no_sorry.py`, the strict audit `lake env lean scripts/AxiomAudit.lean`, and `lake env lean scripts/PrintAxioms.lean`

(`#print axioms` for the headline theorems); it exits non-zero on any failure and ends with `VERIFY OK`. The *Lean certificate* CI workflow runs the same script. `scripts/AxiomAudit.lean` computes `collectAxioms` for every non-internal declaration in the `FrohmanianTether` modules and raises an error if any axiom other than the three standard ones occurs (this includes `sorryAx`, produced by `sorry`, and `Lean.ofReduceBool`, produced by `native_decide`) or if any axiom is declared. `scripts/check_no_sorry.py` rejects the tokens `sorry`, `admit`, `native_decide` and `axiom` declarations outside comments and strings.

`#print axioms` output (`scripts/PrintAxioms.lean`) and audit summary:

```
'ConditionalRegularity.frohmanian_tether_theorem' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'TetheredLyapunov.conditional_regularity' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
'FrohmanianTether.ClayStatements.frohmanian_conditional_regularity_of_closing_bound' depends
↪  on axioms: [propext, Classical.choice, Quot.sound]
'FrohmanianTether.ClayStatements.conditional_regularity_nonvacuous_at_zero' depends on axioms:
↪  [propext, Classical.choice, Quot.sound]
'FrohmanianTether.tetherKernel_C3' depends on axioms: [propext, Classical.choice, Quot.sound]
'FrohmanianTether.pairsToZero_of_C2' depends on axioms: [propext, Classical.choice, Quot.sound]
'FrohmanianTether.uniqueness_of_minimal_tether' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
'FrohmanianTether.uniqueness_of_minimal_symmetric_tether' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'FrohmanianTether.uniqueness_of_minimal_tether_core' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'TetherNondegeneracy.tetherKernel_nondegenerate' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'TetherWeight.not_aeEnstrophyWeightUnrestricted_tetherKernel' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'TetherWeightProof.tetherKernel_aeEnstrophyWeight' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'TetherWeightProof.tetherKernel_core_inputs' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
'FrohmanianTether.not_tetherKernel_invariant_concrete' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'FrohmanianTether.not_tetherKernel_invariant_transSubgroup' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'FrohmanianTether.tetherKernel_transported_invariant' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'FrohmanianTether.not_transported_invariant_pointEval' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'ExplicitFlows.shearFlow_navier_stokes' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
'ExplicitFlows.abcFlow_navier_stokes' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
'ExplicitFlows.shearFlow_isPeriodicSmoothSolution' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'ExplicitFlows.abcFlow_isPeriodicSmoothSolution' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'ExplicitFlows.curl_abcProfile' depends on axioms: [propext, Classical.choice, Quot.sound]
'ExplicitFlows.not_integrable_sq_of_periodic' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
'ExplicitFlows.isEmpty_katoLocalWitness_of_periodic' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
AUDIT SUMMARY: theorems audited: 760 (670 stated in the sources, 90 auto-generated
  equation/congruence/injectivity lemmas); declarations audited: 1074; violations: 0
AUDIT PASS: only propext, Classical.choice, Quot.sound; no sorryAx; no axiom declarations
```

Licence

Copyright © 2026 Benjamin Stanley Frohman. The manuscript and the Lean formalization are released under the Apache License, Version 2.0.

References

[1] V. I. Arnold, Sur la géométrie différentielle des groupes de Lie de dimension infinie et ses applications à l'hydrodynamique des fluides parfaits, *Ann. Inst. Fourier* 16 (1966), 319–361.

[2] V. I. Arnold, B. A. Khesin, *Topological Methods in Hydrodynamics*, Springer, 1998.

[3] J. T. Beale, T. Kato, A. Majda, Remarks on the breakdown of smooth solutions for the 3-D Euler equations, *Comm. Math. Phys.* 94 (1984), 61–66.

[4] C. L. Fefferman, Existence and smoothness of the Navier–Stokes equation, in *The Millennium Prize Problems*, Clay Math. Inst., 2006, 57–67.

[5] P. Isett, A proof of Onsager's conjecture, *Ann. of Math.* 188 (2018), 871–963.

[6] T. Kato, Strong L^p-solutions of the Navier–Stokes equation in $\mathbb{R}^m$, with applications to weak solutions, *Math. Z.* 187 (1984), 471–480.

[7] P. J. Morrison, A paradigm for joined Hamiltonian and dissipative systems, *Physica D* 18 (1986), 410–419.